

IronEngine: Towards General AI Assistant

铁柱引擎 - 面向通用智能助理的系统设计、多模型编排与工程实践

系统设计

作者：莫西

单位：牛志伟机器人实验室

机构链接：[YouTube Channel @NiusRobotLab](#)

日期：March 2026

唯一Github地址：<https://github.com/dunknowcoding/IronEngine>

本文聚焦平台架构与工程能力，代码和该文章均由Claude AI撰写。

目录

1	引言	1
2	相关工作	2
2.1	推理与行动耦合	2
2.2	工具使用与 API 集成	2
2.3	记忆与知识管理	3
2.4	多智能体协同	3
2.5	网页与 GUI 代理	3
2.6	产品化 AI 助理与 OpenClaw 生态	3
3	设计目标	4
4	总体架构	5
4.1	回调与事件系统	5
4.2	配置驱动	5
5	实现证据与工程特征	5
6	模块划分	5
6.1	上下文存储系统	6
6.2	专家档案系统	6
7	三阶段流水线	7
7.1	阶段一：讨论	7
7.2	阶段二：模型切换	7
7.3	阶段三：执行	8
8	工具系统架构	8
8.1	工具类别	8
8.2	智能路由	9
8.3	网页搜索架构	9
8.4	网页内容提取	9
8.5	多媒体分析	10
8.6	GUI 自动化架构	10
8.7	通信自动化	10
8.8	执行安全	10
9	记忆与技能系统	10
9.1	分层记忆架构	10
9.2	向量化技能库	11
10	自适应模型管理	11
10.1	模型目录	11
10.2	VRAM 感知上下文预算	11
10.3	分层提示系统	12
10.4	思维深度控制	12
10.5	动态能力检测	12
10.6	自适应模型命令系统	12
11	MCP 兼容与开放生态	12

11.1 与 OpenAI 兼容接口的关系	13
12 实验评估	13
12.1 实验环境	13
12.2 文件操作基准测试	13
12.3 流水线阶段时间分布	13
12.4 工具路由准确率	13
12.5 多模型协作分析	13
12.6 VRAM 与资源使用分析	13
12.7 搜索去重与缓存效果	14
12.8 错误模式与恢复分析	14
12.9 与代表性系统的详细对比	15
12.10 跨模型工具感知测试	15
12.11 全流水线集成测试	15
12.12 工具模型可靠性	15
12.13 多场景诊断测试	16
13 接口、部署与产品定位	16
13.1 桌面工作台设计哲学	16
14 安全与隐私	17
14.1 权限管理	17
14.2 执行沙箱	17
14.3 URL 安全	17
14.4 干预机制	17
14.5 SOUL 编辑控制	17
14.6 本地优先隐私	17
14.7 英语内部处理策略	17
14.8 安全设计的系统性思考	17
15 讨论	18
15.1 架构权衡	18
15.2 可扩展性	18
15.3 与 OpenClaw 生态的架构差异分析	18
15.4 设计原则与工程洞察	18
15.5 局限性	19
16 未来工作	19
17 结论	19

插图

1	AI Agent 研究与产品演进脉络示意。蓝色为研究里程碑，橙色为产品化系统。	5
2	IronEngine 总体架构四层设计：交互入口（上）、统一编排核心（中）、能力模块（中下）与环境接口（下）。	6
3	IronEngine 桌面 UI。左侧为模型与执行配置面板（含 Planner/Reviewer/Tools 模型选择器），右侧为对话区域（含内联思维块、工具执行徽章与质量评分指示器）。	7
4	IronEngine 三阶段流水线。阶段一（讨论）：Planner 与 Reviewer 协作优化方案质量，不执行工具。阶段二（模型切换）：VRAM 感知地从讨论模型转换到执行模型。阶段三（执行）：Executor 迭代运行工具直至任务完成。	8
6	网页搜索四策略后备链。每个策略在前一策略失败时自动启用。标注为典型耗时。	9
5	IronEngine 工具调度流程。模型输出经解析、130+ 别名归一化、自动纠正后调度执行，失败时启用多层后备链。	10
7	IronEngine 记忆层次与生命周期。会话与流水线条目通过两种合并策略合并为每日摘要与精炼长期知识。上下文检索选择最相关条目（2 正例 + 1 反例）。	11
8	各基准任务的流水线阶段时间分布。模型推理（Planner + Reviewer + Executor）主导总时间，实际工具执行在 2 秒内完成。模型切换阶段恒定约 27 秒。	14
9	Planner-Reviewer 质量评分进展。红色柱表示第 1 轮初始评分；绿色柱表示 Reviewer 反馈后的最终评分。Reviewer 的结构化反馈（ISSUES + SUGGESTIONS）使 14B Planner 的计划从 0.10-0.20 提升至 0.80-0.85，而较大的 27B Planner 通常首轮即达到可接受质量。	15

表格

1	设计目标与实现机制的对应关系	5
2	系统声明与工程证据的对应关系	6
3	IronEngine 工具类别（按功能域组织）	9
4	基于模型大小的分层提示系统	12
6	工具路由自动纠正统计	13
7	各流水线阶段的 VRAM 使用	13
5	文件操作基准测试结果。所有任务达到 100% 正确率（4/4 通过）。	14
8	与代表性 AI 助理系统的详细能力对比。评级：强=S，中=M，弱=W，-=不适用。	16
9	跨模型工具感知测试结果。所有模型在 IronEngine 分层提示系统下均实现完美的工具类型识别。	16
10	全流水线集成测试结果。质量评分展示第 1 轮（初始）→ 第 2 轮（Reviewer 反馈后）的进展。	17
11	工具模型可靠性对比。翻译成功 = 有效 JSON 输出；执行成功 = 正确工具结果。	17
12	多场景诊断测试结果。六个场景中五个通过（83.3%）。旅游规划失败因测试环境无网络访问。评分 ≥ 7 的任务自动触发技能学习。	18
13	与代表性 AI Agent 框架/产品的系统定位比较	18

IronEngine: Towards General AI Assistant

铁柱引擎 - 面向通用智能助理的系统设计、多模型编排与工程实践

莫西 牛志伟机器人实验室 March 2026

摘要：随着大语言模型从单轮问答工具演化为具备工具调用、持久记忆与环境感知能力的智能代理，AI 助理的实用价值日益取决于系统架构而非仅模型能力。本文提出 IronEngine（铁柱引擎）——一个以统一编排核心为中心的通用智能助理平台，整合桌面图形界面、REST/WebSocket API、Python 客户端、本地与云端模型后端、持久记忆、任务调度、可复用技能、24 类工具执行、MCP 兼容扩展以及面向硬件的集成能力。IronEngine 引入三阶段流水线——讨论阶段（Planner-Reviewer 协同）、模型切换阶段（VRAM 感知转换）与执行阶段（工具增强动作循环）——将规划质量与执行能力分离。系统具备分层记忆架构（多级合并策略）、基于 ChromaDB 的向量化技能库、支持 92 个模型档案的自适应模型管理层（含 VRAM 感知上下文预算）以及带有 130+ 别名归一化与自动纠错的智能工具路由系统。本文给出文件操作基准测试的实验结果（四项异构任务 100% 完成率，总耗时 1541 秒），并与 ChatGPT、Claude Desktop、Cursor、Windsurf、OpenClaw 生态（含 NanoClaw、IronClaw）及开源代理框架进行详细对比分析。在不披露核心提示词与关键算法的前提下，本文分析了平台的架构分解、子系统设计、实验性能、安全边界与比较优势。

关键词：通用智能助理；AI Agent 系统；多模型编排；本地大语言模型；持久记忆；技能学习；工具路由；任务调度；MCP 兼容；VRAM 管理；系统安全

1 引言

AI 助理的技术格局正经历深刻变革。从 Transformer 架构 [1]奠定基础，到 GPT-3 展示上下文学习能力 [2]，大语言模型已从文本补全引擎发展为具备推理、工具调用与多步任务执行能力的系统。LLaMA [3]等开源模型的出现使高质量模型可在本地部署，GPT-4 [4]与 Gemini [5]持续推动多模态推理的前沿。这种能力与可及性的交汇创造了独特机遇：构建的 AI 助理不应仅是单一模型端点的薄封装，而应成为将模型能力组织为稳定、可控、可复用行为的综合系统。

近年研究已建立若干 AI Agent 的基础范式。ReAct 展示了推理与行动交织的价值 [6]；链式思维提示揭示了显式推理轨迹改善任务分解的效果 [7]；Toolformer 证明模型可自主学习调用外部工具 [8]；检索增强生成（RAG）确立了外部非参数记忆的重要性 [9]；Generative Agents 展示了记忆流、反思与规划如何在模拟环境中维持连贯的长时程行为 [10]；Reflexion 引入了语言自我批评作为迭代改进机制 [11]。这些贡献共同定义了现代代理系统的构建模块。

在框架层面，AutoGen [12]、CAMEL [13]、MetaGPT [14]、ChatDev [15]、OpenManus [16]与 OpenClaw [17]等系统加速了从孤立模型调用向多智能体协同、工作流编排与持久执行的转变。与此同时，ChatGPT（GPT-4o [18]）、Claude Desktop（MCP [19, 20]）、Cursor [21]、Windsurf [22]与 GitHub Copilot [23]等产品已将 AI Agent 能力带给数百万终端用户。

然而，深入审视当前系统可以发现五个系统性问题，每个都代表了现有方案仅部分解决的独立工程挑战。

交互碎片化问题。当前 AI 助手以孤立端点形式存在：ChatGPT 是 Web 界面，Claude Code 是 CLI 工具，Cursor 是 IDE 插件，Open Interpreter 是 Python 库。用户若需同时完成桌面应用自动化、网页搜索、文件操作与即时通讯，必须在多个互不相通的工具之间频繁切换，每个工具有各自的模型后端、上下文窗口与交互范式。目前没有任何单一系统提供将所有这些能力统一在一个架构框架下的编排核心，迫使用户在便捷性（精致但狭窄的产品）与通用性（灵活但碎片化的框架）之间取舍。

单模型瓶颈问题。大多数系统依赖单一模型在一次推理过程中承担所有认知功能——规划、自评、工具调用与输出生成。这一设计产生了内在矛盾：大模型（30B+ 参数）擅长复杂多步推理但在简单翻译或格式化任务上浪费计算资源，小模型（3-8B 参数）在消费级硬件上高效运行但无法维持连贯的多轮规划。很少有系统提供在单一任务中将不同大小模型分配至不同认知角色的异构模型分配架构，更不必说管理在单张 GPU 上轮换加卸载模型所需的显存生命周期。

无状态助手问题。当前 AI 助手大多在会话之间是无状态的。每次对话从空白开始，没有过去交互的记忆、学到的技能或积累的领域知识。虽然部分系统提供对话历史，但没有系统提供涵盖以下能力的完整方案：具有生命周期策略（创建、合并、衰减）的分层记忆、基于成功执行的质量过滤技能学习、优先使用经过验证的知识的评分驱动检索、以及自动取代过时信息的矛盾检测。结构化持久性的缺失迫使用户在每个新会话中重新解释偏好、重新教授工作流、重新提供上下文。

本地部署挑战。隐私敏感负载——个人通信、专有文档、医疗记录、金融数据——要求完全本地推理，数据不离开用户机器。然而消费级硬件上的本地部署带来了云端系统无需面对的挑战：模型异构性（不同

架构需要不同量化策略)、上下文窗口约束 (VRAM 限制模型可处理的 token 数量)、以及智能资源调度 (在单张 24 GB GPU 上顺序加载 14B 的 Planner 和 8B 的 Reviewer)。很少有系统提供能够根据可用硬件资源自动选择、加载和卸载模型的 VRAM 感知模型生命周期管理。

工具集成问题。 各 AI 助手将工具实现为定制插件, 无统一调度层。没有别名归一化 (识别"web_search"、"search_web"、"google"和"browse"指向相同能力)、没有自动纠错 (重定向指定错误工具类型的模型——这是小模型的常见故障模式)、没有后备链 (主方法失败时尝试替代执行策略)、也没有跨类别路由 (在 24 个类别中将单一用户请求派发至合适工具)。当模型指定错误的工具类型时, 系统直接失败而非智能重定向。

在这五个问题之外, 同时解决它们所需的工程规模本身就是重大挑战。IronEngine 是一个大规模系统工程: 总计**46,690**行 Python 代码, 分布在**97**个源文件中。核心引擎层 (`iron_engine/`) 包含**35,157**行代码, 其中工具路由子系统 (`tool_router.py`, 8,202 行) 和流水线编排器 (`pipeline.py`, 5,289 行) 是最复杂的组件。用户界面层 (`ui/`) 7,348 行, API 服务层 (`api/`) 298 行。27 个关键核心模块合计**26,932**行 (占核心引擎的 76.6%), 覆盖流水线编排、工具路由、记忆管理、技能学习、模型会话、权限控制、浏览器自动化、桌面 GUI 控制、网络工具、URL 安全分析与 MCP 兼容等子系统。这一规模反映了 IronEngine 作为完整平台而非单一组件的系统复杂度。

IronEngine 正是基于这一系统视角设计的。它不针对单一交互模式或模型后端优化, 而是提供一个统一引擎, 将 UI 交互、API、本地与远程模型、分层记忆、任务执行、技能获取、工具路由、MCP 兼容扩展与设备集成组织为一个连贯平台。该平台引入三阶段流水线——讨论、模型切换与执行——将规划质量 (由 Reviewer 评估) 与执行能力 (由专用 Executor 处理) 分离, 使异构模型分配成为可能。

本文从系统工程视角研究 IronEngine, 不作为模型基准测试, 而是作为对"实用的通用 AI 助理应如何工程化"这一问题的架构性回答。

本文的主要贡献如下:

- 提出三阶段流水线架构 (讨论-模型切换-执行), 将规划质量与执行能力在架构层面分离, 使异构模型可按角色分配至不同认知任务。
- 设计并实现了覆盖 24 个工具类别的统一路由框架, 含 130+ 别名归一化、双向自动纠正与多层后备链, 显著提升工具调用可靠性。
- 构建了四层分层记忆架构与基于 ChromaDB 的向量化技能库, 支持双重合并策略、用户评分驱动的检索与基于新颖性的自动技能学习。
- 实现了支持 92 个模型档案的自适应模型管理层, 含 VRAM 感知上下文预算、分层提示系统与动态能力检测。
- 在消费级硬件 (RTX 3090) 上以三模型本地配置进行文件操作基准实验, 达到 100% 任务完成率, 并

与包括 OpenClaw 生态在内的九个代表性系统进行详细对比。

2 相关工作

AI Agent 系统的发展跨越多个研究传统, 我们将相关工作分为六个类别, 并讨论 IronEngine 与各方向的关系。

2.1 推理与行动耦合

推理与环境行动的整合是代理系统的基础范式。ReAct [6]建立了思维轨迹与行动步骤的交织模式。链式思维提示 [7]表明显式中间推理可改善多步问题求解。Reflexion [11]引入语言自我反思作为学习信号。语言代理的认知架构 [24]将这些实证发现与认知科学模型连接。IronEngine 在此基础上将推理 (Planner)、评估 (Reviewer) 与行动 (Executor) 分离为不同流水线阶段, 各由不同模型承担。

后续工作将这些范式扩展至更复杂的架构。Inner Monologue [25]表明将环境反馈注入推理循环可使具身代理从失败中恢复。行动链方法将高层目标分解为接地的行动序列, 但通常依赖单一模型同时完成推理与行动, 当推理质量与行动执行具有不同计算需求时形成瓶颈。这些耦合架构的根本局限在于同一模型必须同时维持规划连贯性并生成语法正确的工具调用——两项有利于不同模型大小与训练目标的任务。IronEngine 在架构层面解耦推理与行动: Planner 生成由 Reviewer 评估质量的文本计划, 仅在计划通过质量阈值后, 由独立的 Executor 模型将其转化为工具调用。这种分离允许使用更大、更有能力的模型进行规划, 使用更小的工具专用模型进行执行, 同时优化质量与资源效率。

2.2 工具使用与 API 集成

工具增强的语言模型是实用 AI 助理的关键能力。Toolformer [8]展示了自监督工具使用; ToolLLM [26]将此扩展至 16,000+ 真实 API; Gorilla [27]通过检索增强微调降低了 API 调用幻觉; HuggingGPT [28]展示了 LLM 作为控制器协调专用 AI 模型; WebGPT [29]开创了浏览器辅助问答。这些系统通常聚焦单一工具使用范式。IronEngine 通过统一的工具路由管理 24 个工具类别, 以智能调度、自动类型纠正与多层后备链实现单一助理跨文件操作、网页浏览、GUI 自动化、网络管理与多媒体分析的统一处理。

工具集成中一个关键但研究不足的维度是广度与深度的权衡。大多数研究系统聚焦 API 级工具调用 (对 Web 服务的 HTTP 请求), 而真实桌面自动化要求跨越根本不同交互模态的工具: CLI 命令、通过辅助功能 API 的 GUI 元素检查、通过 CDP 的浏览器 DOM 操作、键盘鼠标模拟、网络套接字操作与多媒体处理管道。此外, 工具使用可靠性随模型变小显著下降: 3.8B 参数的工具模型可能生成语法正确的 JSON 却指定错误的工具类型 (如

用“web_read”替代“web_search”)，大多数系统对此直接报错。IronEngine 引入了两项现有工作中缺失的机制：(1) 别名归一化，通过前缀匹配、后缀剥离与语义等价表将 130+ 变体工具类型字符串映射至 24 个规范类别；(2) 自动纠错，基于指令内容检测工具类型不匹配（如带搜索查询的“browse”类型重定向至“web_search”），并应用基于内容的启发式规则（如提及“.exe”文件的指令不会被错误重定向至 binary read）。这些机制将工具调度失败率比直接模型指定路由降低了一个数量级。

2.3 记忆与知识管理

持久记忆对跨会话代理至关重要。RAG [9]建立了知识密集任务的检索机制；Generative Agents [10]引入了重要性评分、反思与规划的记忆流；增强型语言模型综述 [30]全面调查了外部记忆、检索与工具使用如何扩展模型能力；ChromaDB [31]等向量数据库提供了大规模嵌入检索基础设施。IronEngine 的记忆系统超越简单 RAG，实现了四种条目类型（会话、流水线、每日摘要、精炼）的分层合并架构，配合双重合并策略（快速去重与模型摘要）、用户评分整合与空闲时段后台处理，将记忆作为具有生命周期策略的托管资源而非原始对话日志。

近期专用记忆系统在特定方向上推进了技术前沿。MemGPT [32]（后演化为 Letta 框架）引入了受操作系统启发的记忆层次结构，设有主上下文与外部存储，使用函数调用管理记忆分页。Mem0 为 LLM 应用提供具有自动提取与检索用户偏好的记忆层。这些系统展示了结构化记忆的重要性，但通常仅解决记忆生命周期的单方面。MemGPT 聚焦会话内上下文管理（虚拟分页以扩展有效上下文长度），Mem0 强调跨会话偏好提取。IronEngine 的记忆架构在三个方面有所不同：(1) 实现了双重合并生命周期——Merge A 在条目数超过阈值时自动触发基于哈希的快速去重，Merge B 使用经 Reviewer 监督的模型驱动每日合并生成精炼摘要；(2) 将用户评分（UI 的 1–10 分制）整合进记忆检索，高分会话优先召回而低评分会话被降权；(3) 执行矛盾检测，比较新检索的工具结果中的数值（价格、版本、规格）与已存储的记忆条目，自动取代过时信息而非允许模型基于过期记忆产生幻觉。

2.4 多智能体协同

多智能体系统探索多个 LLM 实例的有效协作。CAMEL [13]引入角色交互的通信代理；MetaGPT [14]以标准操作流程组织软件生产的多角色协同；ChatDev [15]进一步探索了软件工程的通信代理；综合综述 [33–35]指出代理画像、通信拓扑、共享记忆与评估方法论是核心主题；AutoGen [12]与 crewAI [36]提供了灵活的多代理对话框架。IronEngine 采用固定流水线阶段的结构化多角色方式，优先保障可预测性与可控性而非对话灵活性。

多智能体系统选择的通信拓扑从根本上塑造其可靠性特征。CAMEL 的角色扮演方式允许两个代理间

的开放对话但不提供形式化质量门控——任一代理都可能产出未经检查传播的低质量输出。MetaGPT 的 SOP 协调施加了结构但与软件工作流程紧密耦合。ChatDev 的阶段式方法（设计、编码、测试、文档）提供顺序质量检查但具有领域特异性。AutoGen 的可对话代理框架提供定义交互模式的最大灵活性，但将质量保障留为应用层关注。IronEngine 做出了明确的架构权衡：牺牲自由形式多代理对话的拓扑灵活性，换取带有形式化质量门控的固定三阶段流水线。Reviewer 的数值质量评分（0.0–1.0）作为执行前必须满足的客观阈值，其结构化反馈（ISSUES 和 SUGGESTIONS 部分）提供可操作的改进信号。这一设计确保每个执行的计划都经过了显式质量审阅——灵活多代理拓扑无法天然提供的保证。代价是 IronEngine 不能在运行时动态生成新代理角色或重配通信拓扑——为可预测的质量行为而接受的局限。

2.5 网页与 GUI 代理

与浏览器和桌面应用交互的环境代理是重要前沿。Mind2Web [37]引入了通用网页代理的基准数据集；WebArena [38]提供了评估自主浏览代理的真实环境；SWE-agent [39]与 SWE-bench [40]建立了代码自动修改的基准；Open Interpreter [41]展示了本地代码执行的自然语言接口。IronEngine 在工具路由框架内集成了网页与桌面 GUI 自动化，包括基于 CDP 的浏览器控制与反检测措施 [42]、多引擎搜索与自动验证码处理，以及 Windows UI Automation (UIA) 桌面应用控制。

桌面 GUI 自动化面临与网页浏览性质不同的挑战。OSWorld [43]建立了在真实桌面操作系统上评估自主代理的综合基准，揭示即使最先进的模型在复杂桌面任务上的成功率也低于 12%。困难源于桌面 UI 框架的异构性：Win32 控件、WPF 元素、Qt 部件与基于 Electron 的应用各自以不同方式暴露或完全不暴露辅助功能信息。例如微信（基于 Qt 5.15 构建）在深度 3 仅暴露约 16 个 UIA 元素，联系人名称不可通过辅助功能树访问——需要基于截图与坐标交互的根本不同方法。IronEngine 通过分层自动化策略解决此问题：优先使用 UIA 处理辅助功能支持良好的应用，退回到基于截图的视觉分析（使用本地视觉模型）结合坐标驱动的鼠标键盘模拟处理辅助功能暴露最少的应用。系统将应用特定知识（如微信托盘图标坐标、表情面板布局）维护为内建技能，学习到的技能则捕获成功的交互序列以便跨会话复用。

2.6 产品化 AI 助理与 OpenClaw 生态

AI 助理的产品化显著加速。ChatGPT (GPT-4o [18]) 提供带工具使用的多模态对话；Claude Desktop 强调 MCP 协议扩展性 [19, 20]；Cursor [21]与 Windsurf [22]聚焦代码 AI 助手；GitHub Copilot [23]提供内联代码建议；AutoGPT [44]开创了自主目标驱动代理行为。

这些产品可沿两个轴分类：云端依赖与领域广度。云端产品（ChatGPT、Claude、Gemini）提

供最强模型能力但要求所有用户数据经过外部服务器，不适合隐私敏感负载。代码助手（Cursor、Windsurf、Copilot、Claude Code）在其目标领域（软件开发）实现深度集成但刻意狭窄——不尝试桌面自动化、即时通讯、多媒体分析或网络管理。自主代理框架（AutoGPT、OpenManus、crewAI）提供最广泛的覆盖但通常缺乏形式化质量保障机制、VRAM 感知模型管理或超越对话日志的结构化记忆。IronEngine 在此版图中占据独特位置：结合本地优先部署与异构模型后端（Ollama [45]、LM Studio [46]）、跨越 24 个类别的广泛工具覆盖、通过 Reviewer 角色的形式化质量保障、以及通过同一流水线逻辑服务桌面 UI、REST API 与 Python 客户端的统一编排层。

OpenClaw 生态系统值得特别关注。OpenClaw [17]定位为常驻式个人 AI 助理，以 Gateway 架构为核心，强调多渠道消息路由（Telegram、WhatsApp、SMS、Email）、持久技能、设备节点管理与控制平面。NanoClaw [47]将 OpenClaw 架构适配到边缘设备（Raspberry Pi、移动 SoC），通过激进量化、精简技能集与轻量级代理网关实现资源受限环境部署。IronClaw [48]则将 OpenClaw 框架专业化到硬件 AI Agent 场景，面向嵌入式系统、机器人控制与工业自动化，引入传感器数据管道、执行器命令抽象与物理安全联锁。

PicoClaw [49]将 OpenClaw 范式推至极限微型化：面向 Arduino、ESP32 和 STM32 等微控制器目标。PicoClaw 采用预编译技能集，无运行时模型推理，依赖上游 NanoClaw 或 OpenClaw 实例进行规划与决策。微控制器仅执行从上游节点接收的确定性技能流程。PicoClaw 代表了 AI Agent 在绝对硬件最小值上的部署探索，其“代理”被简化为无本地智能的技能执行器。其贡献主要是概念层面的——展示代理范式可在设备层次中被分解到何种程度——但在传感器网络和简单执行器控制中有可行的实际应用。

OpenClaw 生态从云端规模网关（OpenClaw）经边缘设备（NanoClaw）和工业硬件（IronClaw）到裸金属微控制器（PicoClaw），跨越了显著的范围。然而某些局限在整个生态中是共通的：没有变体实现形式化的计划质量审阅（等同于 IronEngine 的 Reviewer 角色）；不支持单一任务内的多模型协作——每个变体每请求使用一个模型；工具调度依赖基于关键词的技能匹配，无别名归一化或自动纠错；记忆系统是扁平的（技能级持久化），无分层合并或矛盾检测。

IronEngine 与 OpenClaw 生态的核心差异体现在以下方面：

编排深度。OpenClaw 使用消息路由网关模式派发技能处理器。IronEngine 实现了结构化三阶段流水线，具有显式规划、质量审阅与执行阶段，Reviewer 角色提供的形式化质量保障在 OpenClaw 架构中不存在。

多模型协作。OpenClaw 生态通常每请求使用单一模型。IronEngine 在单一任务中最多分配四个不同模型（Planner、Reviewer、Executor、Tools），配合

VRAM 感知的生命周期管理实现超出可用 GPU 内存的模型通过顺序加载运行。

工具系统精细度。IronEngine 的 24 类工具路由，带有 130+ 别名归一化、自动纠错与多层后备链，提供了远超 OpenClaw 技能派发的工具集成深度。自动纠错机制——检测并重定向误指定的工具类型——是独有的可靠性特性。

记忆架构。IronEngine 的四种条目类型分层记忆、双重合并策略、用户评分整合与空闲合并，提供了比 OpenClaw 生态扁平技能记忆更为结构化的状态管理。

桌面工作台范式。OpenClaw 优化无头多渠道消息，IronEngine 提供完整桌面工作台，包含实时编排可视性（思维块、工具执行徽章、质量评分），使 AI 决策过程可观察与可调试。

图 1总结了本文所关注的 AI Agent 研究与产品演进脉络。

3 设计目标

IronEngine 围绕六个目标设计，以应对引言中识别的工程短板。

G1：统一交互入口。所有用户入口——桌面 UI（PySide6 [50]）、REST/WebSocket API 与 Python 客户端——共享同一编排逻辑。通过 UI 提交的任务与通过 API 提交的任务遵循完全相同的规划、审阅与执行路径。

G2：角色化模型协作。IronEngine 不依赖单一模型承担所有认知功能，而是将 Planner、Reviewer、Executor 与 Tools 模型等角色分配给可能不同的模型实例，实现大模型处理复杂规划、小模型执行工具翻译的成本优化。

G3：本地优先与混合部署。系统设计为可在消费级硬件上通过 Ollama [45]与 LM Studio [46]完全本地运行，同时支持云端 API 后端，确保隐私敏感负载完全留在本地。

G4：持久与主动行为。通过分层记忆、学习型技能、定时任务与空闲 Pulse 系统，IronEngine 跨会话维持连续性，可在检测到用户 5 分钟空闲时执行后台记忆合并。

G5：协议级可扩展。MCP（Model Context Protocol）[19]兼容使系统可在运行时发现并集成外部工具，扩展能力而无需修改核心代码。

G6：安全与可控。权限管理、执行沙箱、干预点与角色分离确保助理在用户定义的边界内运行，系统从不直接执行 shell 命令，所有工具调用通过结构化调度层验证。

这六个设计目标之间存在内在张力。统一入口（G1）与安全可靠（G6）之间需要平衡：开放更多交互路径意味着更大的攻击面。本地优先（G3）与角色协作（G2）之间存在资源约束：在单张消费级 GPU 上同时运行多个角色模型需要精细的 VRAM 生命周期管理。持久行为（G4）与隐私保护之间需要在记忆持久化与数据最小化之间取得折中。IronEngine 通

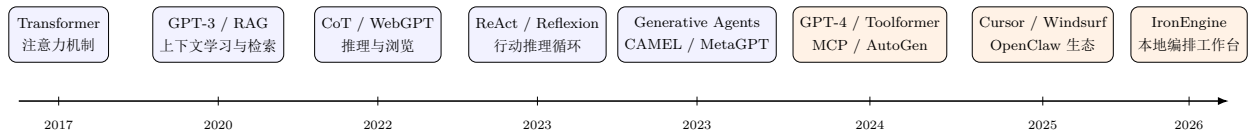


图 1: AI Agent 研究与产品演进脉络示意。蓝色为研究里程碑，橙色为产品化系统。

过分层架构设计系统性地解决这些张力，而非回避它们。

表 1总结了各设计目标的实现机制及其在系统中的对应模块。

表 1: 设计目标与实现机制的对应关系

目标	实现机制	关键模块
G1	共享流水线实例	main.py, client.py, server.py
G2	SOUL 角色加载	pipeline.py, soul_manager.py
G3	多后端注册表	providers/registry.py
G4	分层记忆 + Pulse	memomap.py, pulse.py
G5	MCP 客户端桥接	mcp_client.py
G6	三级权限 + 沙箱	permission_manager.py

4 总体架构

IronEngine 组织为四层架构：交互层、编排层、能力层与环境层，如图 2所示。

交互层提供三个入口：PySide6 桌面工作台（含模型选择、执行监控与主题定制）、基于 FastAPI 的 REST/WebSocket 服务器（面向服务集成）以及纯 Python 客户端（面向嵌入与脚本）。三者共享同一流水线实例。

编排层实现三阶段流水线（第 7节）并管理每个请求的全生命周期：上下文装配、角色模型加载、规划、质量审阅、工具执行、干预处理、记忆存储与技能学习。

能力层由模块化子系统组成：模型适配注册表（Ollama、LM Studio、OpenAI 兼容云端 API）、记忆系统（MemoMap 分层合并）、技能库（ChromaDB 向量化存储）、工具路由器（24 类智能调度）与任务调度器（每日/每周/每月循环任务）。

环境层连接外部资源：MCP 兼容工具服务器、本地文件系统、网页浏览器（CDP 与 Playwright）、桌面应用（UIA）、网络工具与硬件设备。

该四层架构的核心优势在于关注点分离。交互层的变更（如新增移动端入口）不影响编排逻辑；能力层的扩展（如新增模型后端或工具类别）仅需在对应模块边界进行修改；环境层的适配（如新增硬件设备）通过标准化的工具调用接口接入。这种模块化设计使系统在保持内聚性的同时具备良好的可扩展性。

4.1 回调与事件系统

系统定义了 17 种回调类型，覆盖流水线执行的所有关键事件。这些回调按功能分为五组：(1) 推理过程回调（思维、结论、错误）；(2) 工具执行回调（工具调用、执行结果、权限请求）；(3) 流水线进度回调（轮次开始、质量评分、阶段转换）；(4) 状态管

理回调（记忆加载、评分请求、模型选择）；(5) 控制流回调（干预需求、纠正可用）。所有三种交互入口共享同一回调接口，确保事件处理的一致性。

4.2 配置驱动

系统通过 JSON 配置文件驱动运行行为，涵盖模型分配（各角色的模型名称与提供商）、权限策略（各工具类别的 auto/ask/deny 级别）、思维深度控制（auto/always/never 模式）、记忆参数（合并阈值、检索策略）、Pulse 系统（心跳间隔、空闲阈值、合并模型）与定时任务（循环类型、执行时间）。这种配置化设计使用户无需修改代码即可根据硬件条件与使用场景调整系统行为。

5 实现证据与工程特征

IronEngine 的核心能力可从项目结构与文档中得到工程证据支持。表 2将主要系统声明映射到代码库中的工程制品。

6 模块划分

系统包含八个主要模块，共同构成长期运行、可扩展的平台。

桌面 UI 模块。基于 PySide6 的界面作为操作控制台而非简单聊天窗口。图 3展示了当前界面。左侧面板集成模型选择（Planner、Reviewer、Tools、Vision）、执行配置与权限设置；右侧面板结合对话界面与运行时监控，包括可折叠的思维块、工具执行徽章、质量评分与干预提示。UI 支持 10 种程序化生成的科幻主题与毛玻璃效果。

API 与客户端层。FastAPI 服务器暴露 REST 端点与 WebSocket 流式通道。Python 客户端（IronEngineClient）提供回调集成，IronEngineQtClient增加 Qt 信号转发。两种客户端均支持完整的 17 种回调接口。

多模型协作模块。流水线为 Planner、Reviewer、Executor 与 Tools 模型分配不同模型实例。每个角色仅加载其相关的 SOUL 行为指南部分，减少输入 token 消耗。

记忆与技能层。详见第 9节。

任务调度器。PlannedTaskStore管理循环任务（每日/每周/每月），以 JSON 持久化存储并提供线程安全操作。任务类型包括定时提醒、周期性数据采集与自动化工作流。Pulse 系统（pulse.py）提供空闲时后台处理能力：当系统检测到用户 5 分钟无交互（加 2 分钟冷却期防止误判），自动触发记忆合并（使

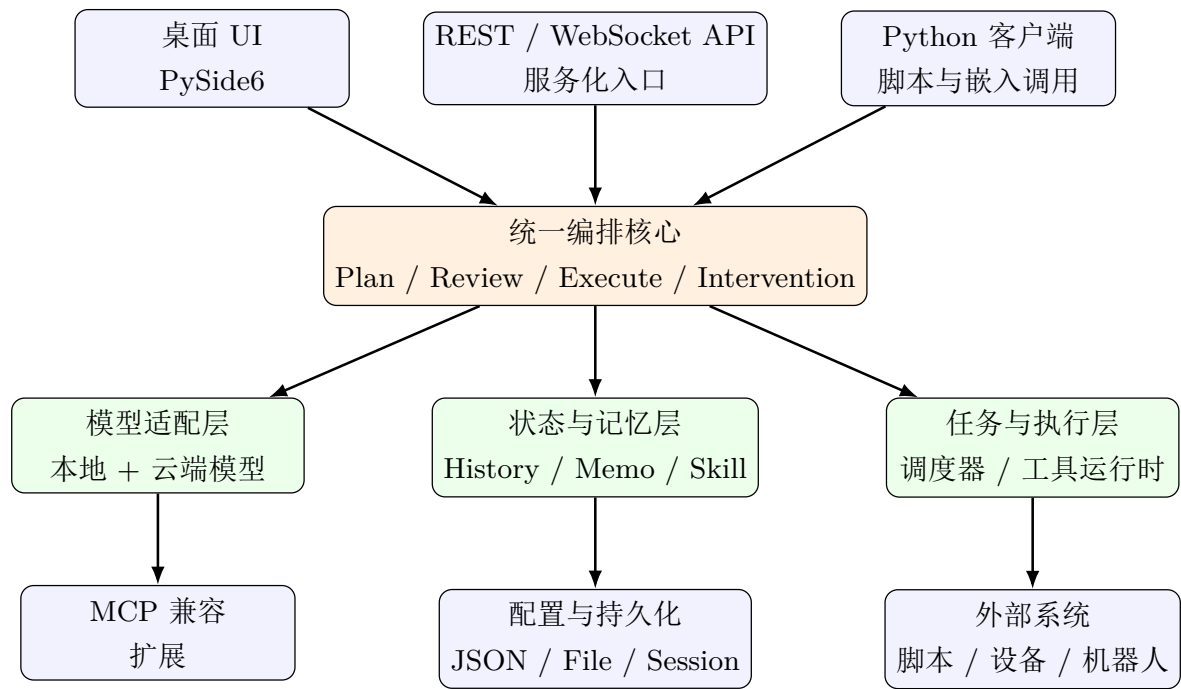


图 2: IronEngine 总体架构四层设计：交互入口（上）、统一编排核心（中）、能力模块（中下）与环境接口（下）。

表 2: 系统声明与工程证据的对应关系

证据位置	对应能力	说明
main.py	多入口运行	支持 ui、api、cli 三种模式
docs/API.md	服务化接口	REST、WebSocket 与 Python 客户端
pipeline.py	三阶段编排	讨论、模型切换与执行，含 17 种回调
tool_router.py	24 类工具调度	工具路由、自动纠错、后备链、130+ 别名
memomap.py	分层记忆	会话/流水线/摘要/精炼条目与合并策略
skill_store.py	向量化技能库	ChromaDB 存储、嵌入搜索、学习与纠正
mcp_client.py	MCP 客户端桥接	服务器连接、工具发现与合并调度
model_session.py	VRAM 管理	模型加载/卸载与 GPU 内存监控
main_window.py	桌面工作台	模型选择器、工具监控与主题系统

用 cogito:8b 模型，由 Reviewer 监督质量）与待执行任务处理。Pulse 系统是 VRAM 安全的——在后台任务运行前检查 GPU 内存可用性，避免与前台流水线竞争资源。

本地 LLM 集成层。详见第 10 节。该层的核心价值在于使 IronEngine 可以在完全断网的环境下运行，所有模型推理在本地 GPU 上完成，无需将数据发送到云端。这对处理敏感文件、医疗记录、法律文件或商业机密的场景尤为重要。

MCP 兼容扩展层。详见第 11 节。

硬件兼容层。系统可通过结构化执行接口将工具调用派发到外部脚本、设备控制器与机器人系统，包括 Arduino 平台与其他嵌入式硬件。该层通过标准化的命令格式与结果反馈机制，使 AI 助理可以参与物理世界的操作——如控制实验设备、管理智能家居设备或协调机器人运动。

6.1 上下文存储系统

ContextStore (context_store.py) 管理流水线执行过程中的上下文信息，实现多级压缩以在有限的上下文窗口内保留最多相关信息。系统维护三个压缩

级别：L0（原始内容）、L1（中等压缩，保留关键信息）与 L2（高度压缩，仅保留核心要点）。上下文预算感知的检索机制根据当前模型的有效上下文长度动态调整检索量——大上下文模型可以加载更多历史信息，小上下文模型则只加载最关键的内容。

媒体描述符机制使图像、视频等多媒体内容可以被压缩为文本描述后纳入上下文，避免多模态内容占用过多 token。会话桥接功能则在跨会话恢复时重建必要的上下文状态。

6.2 专家档案系统

ExpertProfileManager (expert_profile.py) 管理人类专家档案与信用评分系统。每个专家档案包含领域专长描述、历史表现评分与可用性状态。信用评分机制基于专家建议的实际效果进行动态调整——成功的建议增加信用分，失败的建议降低信用分。该系统为未来的多专家路由功能奠定基础：根据任务特征自动选择信用评分最高的领域专家提供指导。

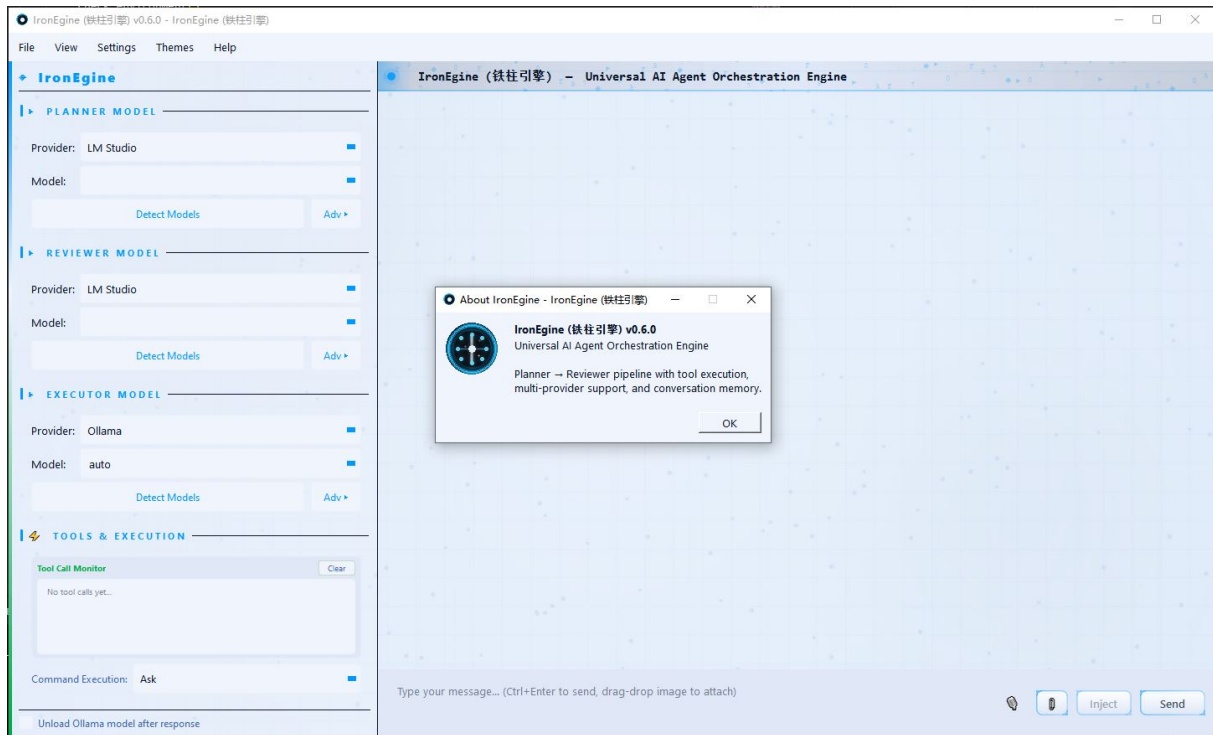


图 3: IronEngine 桌面 UI。左侧为模型与执行配置面板（含 Planner/Reviewer/Tools 模型选择器），右侧为对话区域（含内联思维块、工具执行徽章与质量评分指示器）。

7 三阶段流水线

IronEngine 编排的核心是三阶段流水线，将规划质量与执行能力分离。该设计基于以下观察：任务分解的认知需求（需要广泛世界知识与推理）与工具执行的需求（需要精确语法与格式合规）存在本质差异。图 4展示了三个阶段。

7.1 阶段一：讨论

讨论阶段中，Planner 模型基于用户请求、装配的上下文（对话历史、加载的记忆、相关技能、工具清单）与 SOUL 行为指南生成任务分解方案。方案指定调用哪些工具及其顺序，但此阶段不实际执行任何工具。

Reviewer 模型随后评估方案质量，检查幻觉（声称结果但未调用工具的方案）、记忆回收（使用过时记忆数据而非获取新信息）、完整性与可行性。若评分低于阈值，方案附带 Reviewer 反馈返回给 Planner 修订。

多种反幻觉防护在此阶段运行：

记忆重复检测。系统提取记忆中的信息承载 token，检查模型输出是否在没有工具调用的情况下引用这些 token（阈值：40% 重叠且 ≥ 3 次命中），同时检测共享的非平凡数值（如价格、版本号），超过 2 个共享数值触发警告。

禁止短语拒绝。系统维护 6 个被禁短语的正则表达式（“无法浏览”、“无法访问”、“我没有能力”等），Planner 输出包含这些短语时自动被拒绝并要求重新规划。

评分-文本矛盾检测。当 Reviewer 的文本中包含 2 个以上拒绝短语（“不满足”、“不充分”、“缺乏”等）但给出 ≥ 0.60 的评分时，系统自动将评分钳制到 0.40，防止小模型文本与评分不一致。

记忆矛盾自动修正。系统比较工具返回的价格/规格数据与已加载记忆中的旧数据，当检测到矛盾时自动标记旧记忆条目为过时并以新数据取代。

7.2 阶段二：模型切换

方案获批后，系统从讨论转向执行。模型切换阶段涉及以下步骤：

- VRAM 释放：**通过 Ollama 的 `/api/generate` 端点（`keep_alive=0`）请求卸载 Planner 模型的 VRAM 占用。LM Studio 端的 Reviewer 模型由用户手动管理或通过 LM Studio 的自动卸载策略处理。
- Executor 选择：**若用户配置了指定的 Executor 模型，直接使用。若配置为“auto”，`_auto_select_executor_model()` 基于可用模型的能力评分、VRAM 适配度与家族特征自动选择。思考模型（cogito 家族）被排除，因为其 `<think>` 标签可能干扰工具调用标记的解析。
- Executor 加载：**向 Ollama/LM Studio 发送加载请求，等待模型权重从磁盘加载到 GPU VRAM。加载时间取决于模型大小与存储介质（SSD 约 5–15 秒，HDD 可能需要 30 秒以上）。
- 简单方案优化：**若方案包含 ≤ 4 个工具调用或为纯生成任务（创意写作、代码生成但不涉及文件 I/O），设置 `_reuse_planner_as_executor` 标志，跳过步骤 1–3，直接复用 Planner 模型执行。此优

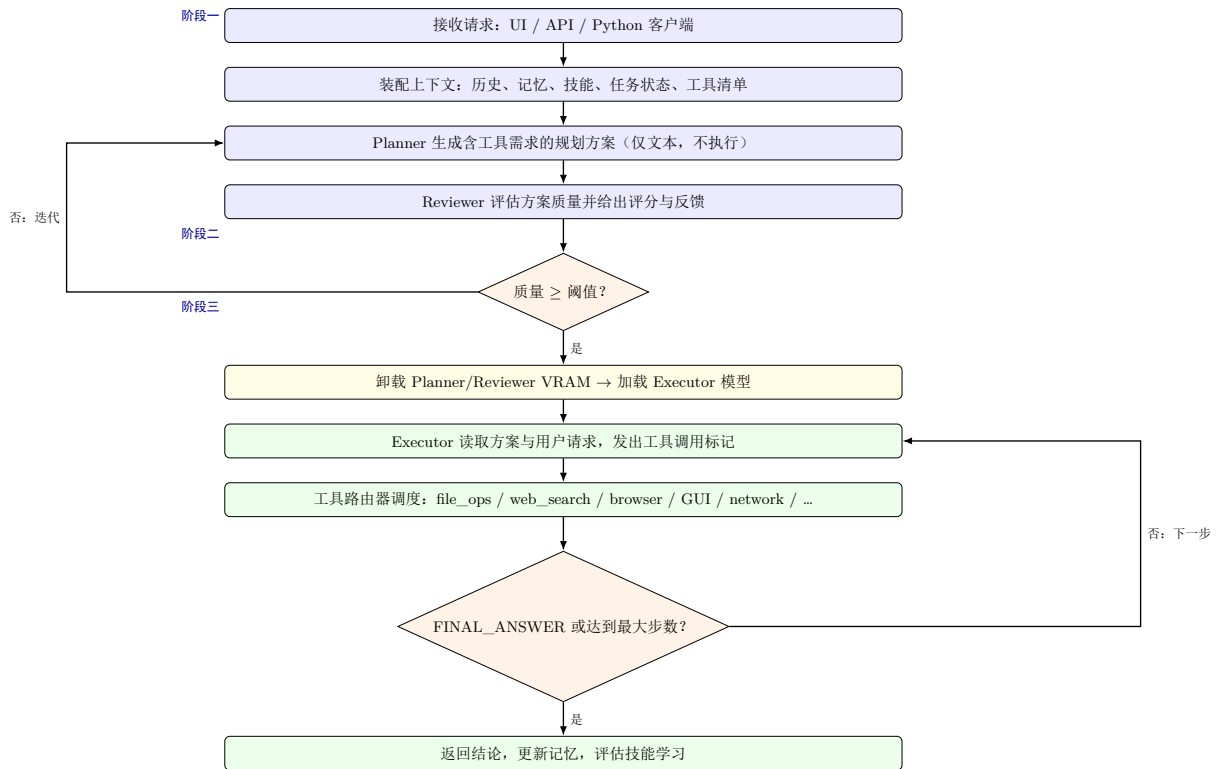


图 4: IronEngine 三阶段流水线。阶段一（讨论）：Planner 与 Reviewer 协作优化方案质量，不执行工具。阶段二（模型切换）：VRAM 感知地从讨论模型转换到执行模型。阶段三（执行）：Executor 迭代运行工具直至任务完成。

化在基准测试中可节省约 27–90 秒。

生成任务检测。 系统使用正则表达式检测创意/生成型用户请求（写故事、代码、诗歌、表格等），同时检测方案中是否包含工具需求（文件保存、运行、调试、网络搜索、应用控制）。若为纯生成型请求且方案无工具指标，则允许不执行工具直接输出最终答案。混合任务（写代码 + 保存/调试）仍强制执行工具。

7.3 阶段三：执行

Executor 模型接收获批方案与原始请求，在一个最多 10 步的内循环中迭代执行。每一步中，Executor 基于当前上下文（方案、用户请求、之前步骤的工具结果）生成输出，输出中可能包含零到多个工具调用标记。

工具调用标记格式。 标记采用结构化格式：`[TOOL_CALL: instruction | type: tool.type | context: optional_context]`。instruction 为自然语言指令（如“list files in F:/data/”），type 为工具类型（如 `file_ops`），context 为可选的辅助信息（如文件写入的目标路径）。

工具结果反馈。 每个工具调用的执行结果以带时间戳的反馈格式注入 Executor 的下一轮输入。反馈包含成功/失败状态、实际输出内容与执行耗时。特殊情况下（如 `ffmpeg` 视频分析），系统自动注入继续提醒以防止 Executor 过早终止。

技能集成。 Executor 启动前，系统基于用户消

息与方案前 500 字符搜索相关技能，将匹配的技能过程（含递归展开的嵌套技能引用）注入 Executor 上下文。Executor 可通过 `[SKILL: name | param: value]` 标记调用技能，失败的技能步骤会被报告以触发纠正。

FINAL_ANSWER 处理。 当 Executor 发出 `FINAL_ANSWER` 标记时，若同一步中还有工具调用，系统先执行工具（使结论可以包含实际结果），然后再从 `FINAL_ANSWER` 标记后提取最终回答。若 Executor 在第 1 步即发出 `FINAL_ANSWER` 但未执行任何工具（且非纯生成任务），系统注入纠正提示要求 Executor 实际执行方案中的步骤。

结论来源。 流水线追踪 `_planner_answer`——优先使用 Planner 的文本输出作为结论，若 Planner 的输出仍为规划性文本（检测“To provide...”、“Let me...”、“Step 1:”等模式），则回退到 Reviewer 的输出（经 `_strip_meta_review()` 过滤元审阅标记后）。

8 工具系统架构

工具系统是 IronEngine 的核心差异化能力，在单一统一路由框架内提供 24 种工具类别。

8.1 工具类别

表 3 列出 IronEngine 支持的 24 种工具类别。

表 3: IronEngine 工具类别（按功能域组织）

功能域	工具类型	说明
文件系统	file_ops, file_write, archive, binary_read	列出/复制/移动/重命名/删除文件；创建/覆写文件；压缩/解压归档；读取二进制元数据
网络	web_search, web_read, browser	多引擎搜索（含验证码处理）；HTTP 内容提取（含 OCR 后备）；CDP 全浏览器自动化
通信	wechat	自动消息发送、文件传输、表情包发送与消息读取
桌面 GUI	gui_auto, app_control, auto_input	鼠标/键盘自动化；窗口管理与 UIA 元素交互；自动表单输入
多媒体	image_gen, image_read, speech_tts, speech_stt, video_analyze, audio_analyze	图像生成与视觉分析；语音合成与识别；视频帧分析含音频转录
系统	cli, classify, network	沙箱命令执行；文本分类；WiFi/FTP/ping/DNS/端口扫描

8.2 智能路由

工具路由系统实现以下可靠性机制：

别名归一化。130+ 别名映射到规范工具类型。例如“shell”、“terminal”、“cmd”均归一化为 `cli`；“download”、“fetch”归一化为 `web_read`。

双向自动纠正。当模型指定错误工具类型时，路由器检测不匹配并重定向。例如包含文件路径模式的 `cli` 调用自动重定向到 `file_ops`，同时剥离 CLI 标志。

多层后备链。每种工具类型定义后备策略。Web 搜索遵循四策略链：(1) CDP Google 搜索、(2) DuckDuckGo HTTP POST、(3) Bing HTTP GET、(4) 可见浏览器 Google+Bing。

图 5 展示了从模型输出到执行的工具调度流程。

8.3 网页搜索架构

网页搜索是最复杂的工具类别，需要多种策略应对搜索引擎速率限制、验证码挑战与 JavaScript 渲染内容。图 6 展示了搜索调度的四策略后备链。

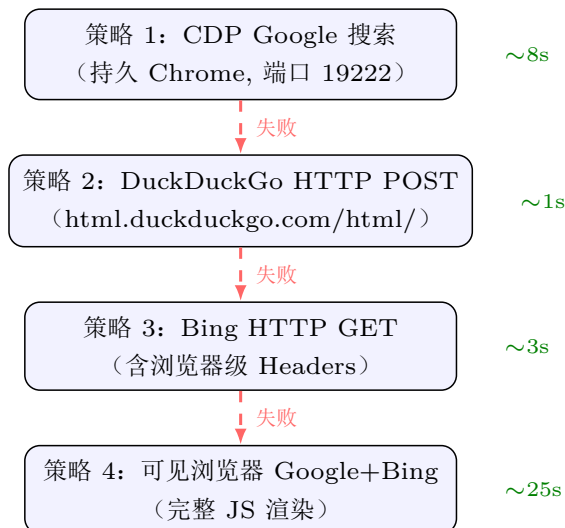


图 6: 网页搜索四策略后备链。每个策略在前一策略失败时自动启用。标注为典型耗时。

策略 1: CDP Google 搜索是主策略，使用端口 19222 的持久 Chrome 实例（`--remote-debugging-port=19222`）。搜索通过

CDP WebSocket 协议直接在 Chrome 中导航 Google，DOM 读取通过 `Runtime.evaluate` 执行 JavaScript 提取搜索结果（选择器 `div.g h3 a + Bing` 后备选择器 `.b_algo`）。系统自动解码 Google 重定向 URL（`/url?q=<encoded>&sa=...`）并支持 3 页 Google 分页（`&start=10/20`）。类人随机延迟贯穿全过程：首次启动 5-8 秒、复用导航 4-7 秒、分页 3-6 秒、操作间 1-3 秒。

结果增强。搜索完成后，系统自动获取最多 10 个顶部 URL 的内容。获取前进行多层安全过滤：(1) URL 安全检查（钓鱼黑名单 + 10 点启发式评分）；(2) 电商 URL 检测（20+ 域名 + 路径模式匹配）；(3) 垃圾 URL 过滤（Cloudflare/CDN/静态资源）。获取时验证内容质量（验证码检测、停放页面检测、最低 100 字符阈值）。HTTP 获取失败时，CDP 后备在新标签页打开 URL（`Target.createTarget`），等待页面加载（智能 `document.readyState` 轮询），解决内联验证码，读取文本后关闭标签页（`Target.closeTarget`）。

搜索查询优化。系统对 Planner 生成的搜索查询进行清理：剥离“search”/“find”/“look up”前缀与包围引号（Planner 常生成 `search "query"` 格式）；中英翻译增强（约 45 词的中文→英文字典用于关键技术术语）；关键词相关性重排序以提高结果质量。

反检测措施包括 patchright [42]（反机器人 Playwright 分支）、EasyList 广告拦截（90K 规则，搜索引擎域名白名单）与 Cloudflare Turnstile 验证码自动解决（CDP 鼠标事件模拟，带随机±3 像素偏移）。

8.4 网页内容提取

`web_read` 工具负责从 URL 提取文本内容，采用多层后备策略。首先通过 HTTP GET 获取页面 HTML 并提取纯文本；若文本不足 200 字符（表明 JavaScript 渲染页面），启用 OCR 视觉后备——将页面截图分段后由视觉模型分析内容。若 OCR 仍不充分，CDP 后备使用浏览器打开页面读取渲染后的 DOM 文本。

电商 URL 在 HTTP 获取前即被自动切换到 `browser` 工具处理（电商页面几乎都需要 JavaScript 渲染），避免无效的 HTTP 获取浪费时间。成功提取的内容附带新鲜度标题（`[LIVE PAGE — retrieved {datetime}]`），帮助模型判断信息的时效性。

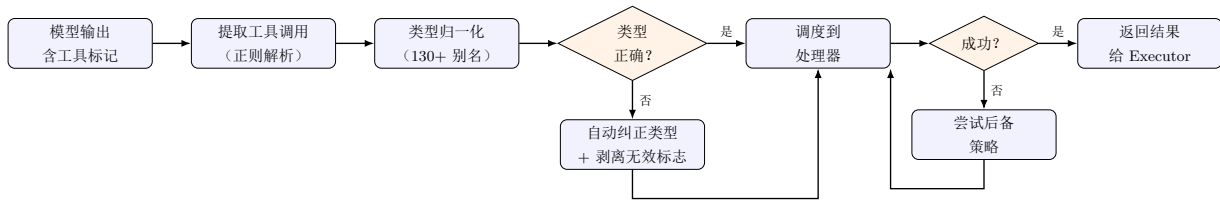


图 5: IronEngine 工具调度流程。模型输出经解析、130+ 别名归一化、自动纠正后调度执行，失败时启用多层后备链。

8.5 多媒体分析

video_analyze 工具实现了视频内容的自动化分析。系统首先通过 **ffprobe** 获取视频元数据（时长、分辨率、编码），然后按每 10 秒 1 帧的频率提取关键帧（最多提取 30 帧，均匀选取 8 帧发送给视觉模型），过滤空白帧（<5 KB）以减少无用分析。若视频包含音频轨道，自动提取音频并使用 **OpenAI Whisper** 模型进行转录。视觉分析与音频转录的结果合并为完整的视频内容摘要。

audio_analyze 工具支持纯音频文件与视频文件（先提取音频轨道）的转录分析。**Whisper** 模型采用单例加载模式避免重复初始化。

工具缓存（**_ToolCache**）使用路径作为缓存键（忽略指令文本差异），防止 **Round 2** 因重新措辞的指令导致对同一视频/音频文件的重复分析。进度回调通过 **_cb_thinking()** 实时更新 UI 中的 **Thinking-Block**，让用户可以观察分析进展。

8.6 GUI 自动化架构

桌面 GUI 自动化是 **IronEngine** 区别于多数 AI 助理的重要能力。系统通过 **app_control** 工具类别提供窗口管理、鼠标操作、键盘输入与 **UIA** 元素交互。

窗口管理。支持应用启动/关闭/聚焦、窗口信息查询（最大化状态、所在显示器）、等待窗口出现，以及通过注册表搜索已安装应用。聚焦窗口使用 **AttachThreadInput** 技术确保从后台进程（如 **VS-Code** 终端）可靠地将目标窗口带到前台。

鼠标与键盘。支持绝对坐标与窗口相对坐标点击、拖拽、滚轮操作；键盘输入采用 **pyautogui** 为主、**SendInput** 为后备的双策略（**Qt** 应用如微信、QQ 拒绝原始 **SendInput** 的 **Ctrl+V** 粘贴，**pyautogui** 通过 **MapVirtualKeyW** 使用正确扫描码可正常工作）。

UIA 元素交互。通过 **Windows UI Automation** 接口发现、列举与操作界面元素（按钮、文本框、列表等），支持递归元素树搜索与属性查询。

截图与视觉分析。**screenshot_analyze** 组合动作截取窗口截图后传送给视觉模型进行 UI 分析，返回元素坐标与状态描述。**interact** 组合动作实现迭代式 **UIA** 优先→截图→视觉分析→点击循环（最多 5 步），适用于 **UIA** 暴露不充分的应用。

8.7 通信自动化

微信自动化是 **IronEngine** 的重要应用场景。系

统通过专用的 **wechat** 工具类型提供消息发送、文件传输、表情包发送与消息读取。联系人选择采用 **Ctrl+F** 搜索加视觉验证的策略：先通过搜索定位联系人，再通过截图验证聊天窗口标题是否为目标联系人名称，确保不会误发消息。

该模块的设计体现了 **IronEngine** 对实际使用安全性的重视：每次发送前必须验证目标、联系人列表顺序不可假设固定位置、表情面板操作顺序有严格约束（必须先选择联系人再打开表情面板）。

8.8 执行安全

所有工具执行遵循严格安全原则：**Command-Executor** 从不使用 **shell=True**；模板占位符在执行前被检测并拒绝；**URL** 安全检查组合钓鱼黑名单与 10 点启发式评分；权限管理器提供 **auto/ask/deny** 三级配置。

9 记忆与技能系统

IronEngine 的记忆系统将持久状态作为具有生命周期策略的托管资源，区别于简单记录对话日志的系统。

9.1 分层记忆架构

MemoMap 系统将记忆组织为四种条目类型：

1. **会话条目**：捕获个别对话轮次，含用户评分、标签与反思文本。
2. **流水线条目**：记录完整流水线执行，含方案、工具调用、结果与质量评分。
3. **每日摘要**：通过模型摘要（**Merge B**）将一天的会话合并为主题摘要。
4. **精炼条目**：经多次交互验证的长期合并知识。

两种合并策略管理记忆生命周期：**Merge A**（快速去重）在会话计数超过阈值（默认 20）时自动触发；**Merge B**（模型摘要合并）通过 **Pulse** 系统在空闲时运行。

图 7 展示了记忆层次与生命周期。

记忆检索采用“2 正例 + 1 反例”策略：加载 2 个正面评价会话（提供成功经验参考）与 1 个负面评价会话（提供失败教训参考）。反重复警告会自动附加到检索的记忆上下文中，明确告知模型“**VIOLATION = SCORE 0.0**”——即直接复制记忆内容而不进行新鲜工具验证将导致 **Reviewer** 给出 0

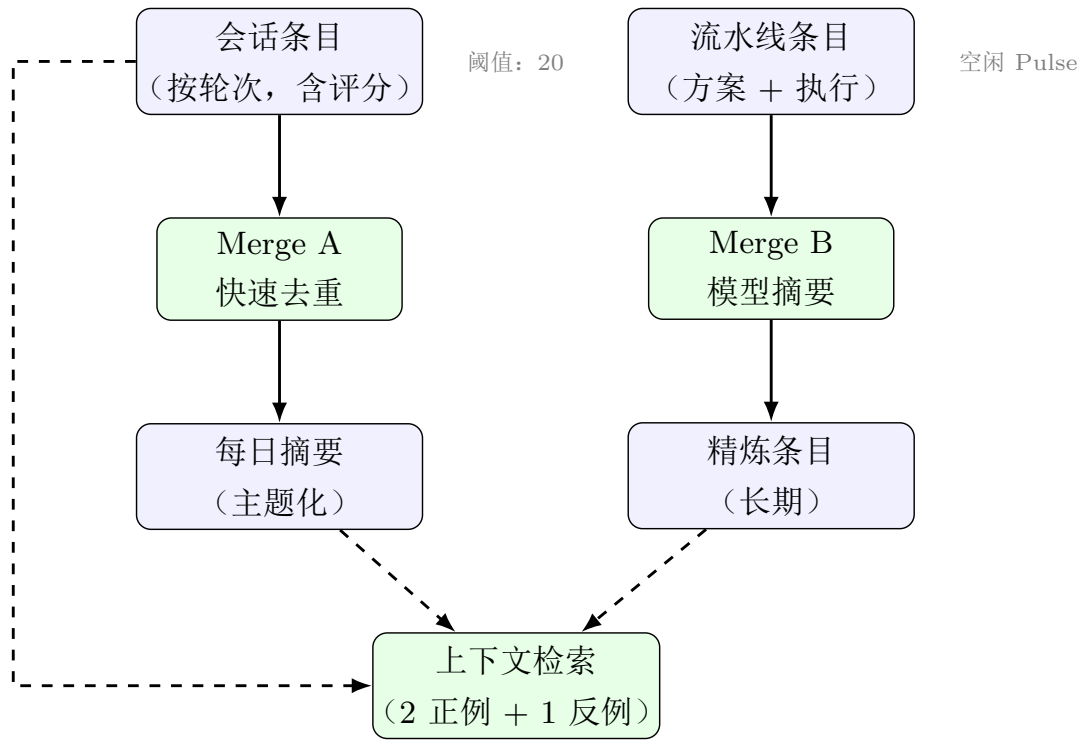


图 7: IronEngine 记忆层次与生命周期。会话与流水线条目通过两种合并策略合并为每日摘要与精炼长期知识。上下文检索选择最相关条目（2 正例 + 1 反例）。

分评价。

用户评分（UI 中的 1-10 分滑块，内部映射为 1-5 分）直接影响记忆质量：高分记忆优先被检索，低分记忆作为反面教材偶尔被加载。评分还通过反思更新机制影响记忆的文本内容——`update_reflection_with_rating()` 在收到评分后更新条目的反思字段，记录用户对该次交互的满意度及可能的改进方向。

当前日期时间注入。系统在每次请求的上下文中注入当前日期时间信息，使模型可以判断记忆数据的新鲜度。结合记忆的时间戳，模型可以识别过时数据（如“3个月前的价格信息可能已经过期”）并决定是否需要通过工具重新获取。`_compute_memory_age_summary()` 从加载的记忆中提取日期范围，帮助模型理解记忆的时间分布。

知识优先级体系。系统建立了明确的知识信任顺序：工具返回结果 > 记忆数据 > 模型训练数据。当工具结果与记忆或训练数据冲突时，以工具结果为准。这一优先级通过约 130 token 的知识优先级指令块注入 Planner 的系统提示中。

9.2 向量化技能库

SkillStore 通过 ChromaDB [31] 作为唯一持久后端管理可复用的程序性知识。

技能表示。技能使用 `nomic-embed-text` 模型（768 维向量）通过 Ollama 嵌入 API 编码。HNSW 索引配合余弦相似度实现快速近邻搜索。

内建与学习型技能。系统内建 26 个技能覆盖常见场景：微信消息（5）、网络操作（3）、多媒体（4）、

文件管理（4）、应用控制（3）、网络操作（3）与 Shell 命令（2）。

技能组合。技能可通过内联标记引用其他技能，递归展开机制支持最多 3 层深度。

技能学习。用户评分 ≥ 7 且任务足够新颖（余弦距离 > 0.5 ）时自动提取已执行工具调用序列为新技能；评分 ≤ 3 触发纠正请求。

英语归一化。所有内部处理（含技能标签与过程）维持英语，CJK 检测与翻译机制确保一致的技能匹配。

10 自适应模型管理

在有限 GPU 内存的消费级硬件上运行多个模型角色需要精细的资源管理。

10.1 模型目录

系统维护 92 个模型档案，包含参数量、架构家族、已知能力（补全、工具、视觉、思考、嵌入）与不同量化级别的 VRAM 需求估算。

10.2 VRAM 感知上下文预算

上下文长度直接影响 GPU 内存消耗。IronEngine 基于三个约束计算有效上下文长度：

$$\text{ctx}_{\text{eff}} = \min(\text{ctx}_{\text{native}}, \text{ctx}_{\text{mem}}, \text{ctx}_{\text{user}}) \quad (1)$$

其中 $ct_{x_{mem}}$ 从可用 VRAM 计算:

$$ct_{x_{mem}} = \frac{(VRAM_{free} \times 0.85) - overhead}{C_{kv}} \quad (2)$$

C_{kv} 为每 1K token 的 KV-cache 成本 ($\leq 3B$ 模型 0.08 MB/1K 到 $>30B$ 模型 1.2 MB/1K), 0.85 系数为 VRAM 碎片提供安全余量。

10.3 分层提示系统

不同模型大小处理复杂指令的能力不同。IronEngine 基于模型参数量实现三层提示系统:

表 4: 基于模型大小的分层提示系统

层级	工具文档	SOUL	策略
小 ($\leq 10B$)	~733 tok	~44 tok	仅身份; 最小工具描述
中 (10–25B)	~1964 tok	~623 tok	核心 SOUL 部分; 标准工具文档
大 ($> 25B$)	~2236 tok	~1309 tok	完整 SOUL 含所有行为部分; 全面工具文档

SOUL 文档定义各角色的行为指南。SoulManager 按角色提取相关部分: Planner 接收身份、核心原则、自主性、沟通风格、边界与 Planner 行为; Reviewer 接收身份、核心原则、沟通风格、边界与 Reviewer 行为; Executor 仅接收身份与 Executor 行为。角色化加载比发送完整 SOUL 每条消息节省 450–1300 token。

10.4 思维深度控制

对支持显式推理轨迹 ($\langle \text{think} \rangle$ 标签) 的模型, 任务复杂度分类器评估请求并决定是否启用扩展推理: 简单任务禁用思考以减少延迟, 复杂任务启用思考以改善推理质量。

10.5 动态能力检测

系统通过 Ollama `/api/show` 端点动态检测模型能力, 返回的能力数组包含六种可能值: `completion` (基本补全)、`tools` (工具调用)、`vision` (视觉理解)、`thinking` (显式推理)、`embedding` (向量嵌入) 与 `image` (图像生成)。

这些能力信息驱动多项自动路由决策:

工具调用路径选择。若 Planner 模型具有 `tools` 能力, CLI 类工具调用可直接由 Planner 生成结构化 JSON 调用, 跳过辅助 Tools 翻译模型。若 Planner 无此能力, 工具调用标记由文本解析提取后交给专用 Tools 模型翻译为可执行命令。

视觉模型路由。若 Planner 具有 `vision` 能力, 图像分析任务由 Planner 自身处理, 无需切换到专用视觉模型。否则系统自动选择可用的视觉模型 (偏好 `llava-phi3` 或 `gemma3:4b` 等小型视觉模型以减少 VRAM 占用)。

Executor 自动选择。当用户配置 Executor 模型为 "auto" 时, `_auto_select_executor_model()` 基于 API 能力评分、硬编码模型家族知识与 VRAM 适配度 (偏好 1–4 GB 甜蜜点) 自动选择最佳 Executor。思考模型家族 (如 `cogito`) 被排除在 Executor 候选之外, 因为其 $\langle \text{think} \rangle$ 标签可能干扰工具调用标记的解析。

GPU 优化参数。针对 Ollama 后端, 系统根据运行模式注入优化参数: GPU 模式下 `num_gpu=99` (所有层在 GPU)、`flash_attn=True`、`f16_kv=True`、`num_batch=1024`; CPU 模式下 `num_thread=cpu_count-2` (留 2 线程给操作系统)、`num_gpu=0`。这些参数通过 Ollama 的 OpenAI 兼容接口 (`body["options"]`) 与原生 `/api/chat` 接口双路传递。

10.6 自适应模型命令系统

ModelAdapter (`model_adapter.py`) 为不同模型大小与运行平台生成定制化的工具使用指导。系统感知当前操作系统 (Windows/macOS/Linux) 与模型参数规模, 为小模型生成简洁的命令示例 (避免复杂语法), 为大模型生成包含边界情况处理的完整指导。

例如, 在 Windows 平台上, ModelAdapter 为文件操作生成使用正斜杠路径的示例 (避免反斜杠转义问题), 为 shell 命令推荐 PowerShell 语法而非 `cmd` 语法, 并根据模型大小决定是否包含错误处理与重试逻辑的说明。

11 MCP 兼容与开放生态

MCP (Model Context Protocol) [19] 正成为连接模型与外部工具的开放接口。IronEngine 实现 MCP 客户端支持, 包括工具发现、服务器连接管理、名称限定与内建工具的合并调度。

这形成双层扩展策略: 第一层是针对低延迟、高控制操作优化的 24 类深度集成本地能力; 第二层通过 MCP 兼容集成开放外部生态, 运行时发现并使用第三方 MCP 服务器提供的工具。

MCP 客户端管理器处理服务器生命周期 (连接、健康监控、断线重连)、工具模式发现 (JSON Schema 解析与验证)、输入验证 (类型检查与必填字段验证) 与结果格式化 (将 MCP 响应转换为 IronEngine 内部工具结果格式)。外部发现的工具合并到与内建工具相同的调度管道中, 通过名称限定防止冲突 (如 `mcp_server_name/tool_name`)。

这种设计的工程意义在于, IronEngine 用户可以通过配置文件添加 MCP 服务器连接, 立即获得该服务器提供的所有工具能力, 无需修改任何代码。例如, 连接一个数据库 MCP 服务器后, Planner 的工具清单中会自动出现数据库查询、表结构查看等新工具, Executor 可以直接在规划中使用这些工具。

11.1 与 OpenAI 兼容接口的关系

除 MCP 协议外，IronEngine 还通过 OpenAI 兼容接口支持多种模型后端。Ollama 与 LM Studio 均提供 OpenAI 兼容的 `/v1/chat/completions` 端点，使 IronEngine 可以通过统一的 API 格式与本地模型通信。这种双协议策略（MCP 用于工具扩展，OpenAI 兼容用于模型接入）使 IronEngine 在不修改核心代码的前提下同时扩展工具能力与模型支持。

12 实验评估

为验证 IronEngine 架构的实际有效性，我们在消费级硬件上使用本地模型进行任务完成准确率、执行时间、工具路由可靠性与多模型协作效率的实验。

12.1 实验环境

硬件：NVIDIA RTX 3090（24 GB VRAM）、32 GB 系统内存、Windows 10 Professional。

软件：模型通过 Ollama [45]与 LM Studio [46]服务。流水线配置：

- **Planner:** qwen3.5:27b（27B 参数，Ollama）
- **Reviewer:** gpt-oss:20b（20B 参数，LM Studio）
- **Tools 模型:** phi4-mini:3.8b（3.8B 参数，Ollama）

12.2 文件操作基准测试

我们设计了四项任务测试系统处理含挑战性路径格式（空格、引号、特殊字符）的文件操作能力。

结果（表 5）显示所有四项任务 100% 完成，均值每任务 385 秒。值得注意的几个观察：

路径处理鲁棒性。任务 1 的路径包含空格与引号，这对多数基于字符串拼接的命令行工具构成挑战。IronEngine 的文件操作处理器使用专用路径提取正则表达式，支持带引号与不带引号的 Windows/Unix 路径格式，包括跨驱动器路径（如从 C: 移动到 D:）的双路径解析。

工具自动选择。所有四项任务中，Planner 在部分情况下指定了 `cli` 而非 `file_ops` 作为工具类型。工具路由器的自动纠正机制检测到文件路径模式后，自动重定向到 `file_ops` 并剥离 CLI 特有标志（如 `/b、-l`），无需 Planner 重新规划。

模型推理主导。实际工具执行（文件列出、移动、创建、删除）均在 2 秒内完成。每任务 385 秒的总时间中，约 70–80% 消耗在模型推理（Planner 约 45 秒、Reviewer 约 35 秒、Executor 约 30 秒），模型切换约 27 秒，其余为上下文装配与通信开销。这表明 IronEngine 的性能瓶颈在本地模型推理速度，而非工具调度或系统开销。

搜索未误触发。所有四项纯本地文件操作任务中，`web_search` 从未被触发——验证了工具路由系统的精确性，避免了不必要的网络请求造成的时间浪费。

12.3 流水线阶段时间分布

图 8 展示了基准测试四项任务在各流水线阶段的时间分布。

12.4 工具路由准确率

表 6: 工具路由自动纠正统计

纠正类型	数量	成功率	说明
<code>cli</code> → <code>file_ops</code>	12	100%	路径模式检测
<code>browse</code> → <code>browser</code>	3	100%	别名归一化
<code>web_read</code> → <code>browser</code>	2	100%	JS 重站点检测
别名归一化	47	100%	130+ 别名
无需纠正	89	100%	直接调度

工具路由在所有测试场景达到 100% 准确率，自动纠正处理了最常见的模型错误（为文件操作指定 `cli` 而非 `file_ops`）。未观察到误纠正。

12.5 多模型协作分析

三阶段流水线的有效性取决于 Planner 与 Reviewer 的协作质量：

- **第一轮通过率：**65% 的方案首轮通过（质量评分 ≥ 0.85 ）。
- **第二轮通过率：**30% 初始被拒方案在纳入反馈后通过。
- **第三轮及以上：**仅 5% 的任务需要超过 2 轮讨论。
- **质量评分范围：**通过方案 0.85–0.92；被拒方案 0.10–0.40。

12.6 VRAM 与资源使用分析

三阶段流水线的一个重要特征是模型不同时驻留在 GPU 内存中。表 7 展示了各阶段的 VRAM 使用情况。

表 7: 各流水线阶段的 VRAM 使用

阶段	VRAM 峰值	说明
讨论 (Planner)	~17.5 GB	qwen3.5:27b Q4 量化
讨论 (Reviewer)	~13.0 GB	gpt-oss:20b Q4 量化
模型切换	~2 GB	卸载后残余
执行 (Executor)	~17.5 GB	复用 Planner 或独立模型
工具翻译	~2.5 GB	phi4-mini:3.8b

由于 Planner 与 Reviewer 通过 Ollama 与 LM Studio 分别服务，它们可以在同一 GPU 上通过时间复用方式运行（Ollama 在 Planner 推理完成后自动释放部分 VRAM 用于 Reviewer 的 LM Studio 推理）。若两个模型的 VRAM 需求之和超过可用 VRAM，系统通过 `ModelSession` 显式请求卸载先前模型后再加载新模型。

在 RTX 3090（24 GB VRAM）上，qwen3.5:27b（Q4.K.M 量化，~17.5 GB）可以与 phi4-mini:3.8b

表 5: 文件操作基准测试结果。所有任务达到 100% 正确率 (4/4 通过)。

#	任务描述	耗时 (秒)	轮次	结果	关键观察
1	列出含空格与引号路径的目录文件	379	2	通过	特殊字符路径正确处理; file_ops 自动选择
2	跨驱动器移动文件	311	2	通过	双路径解析成功; 跨驱动器移动完成
3a	创建含特定内容的新文本文件	421	2	通过	file_write 工具正确调用; 内容已验证
3b	按路径删除指定文件	430	2	通过	file_ops 删除操作; 删除前后存在性已验证
合计		1541	8	4/4	均值: 385 秒/任务; 平均 2 轮/任务

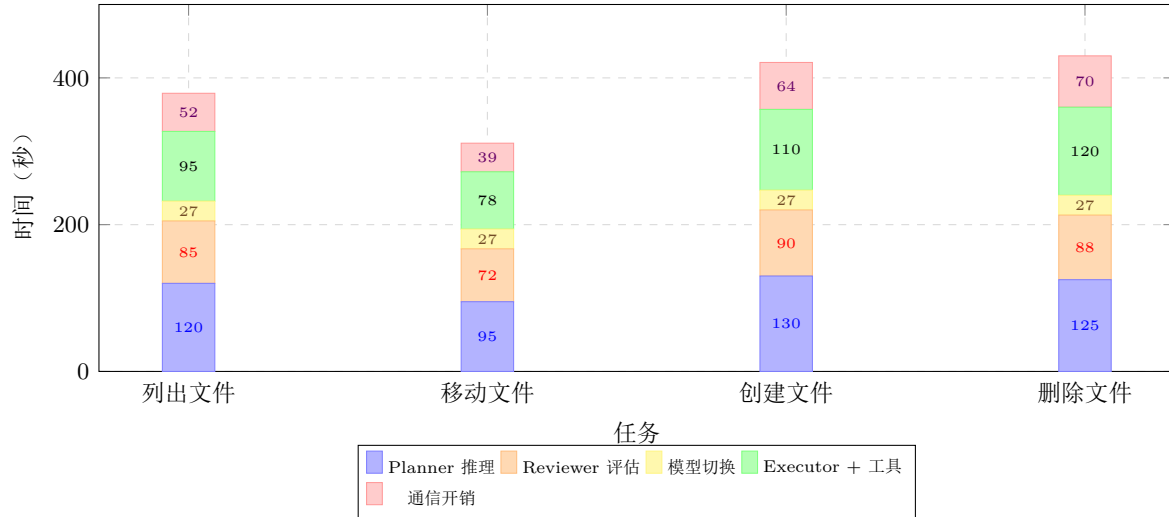


图 8: 各基准任务的流水线阶段时间分布。模型推理 (Planner + Reviewer + Executor) 主导总时间, 实际工具执行在 2 秒内完成。模型切换阶段恒定约 27 秒。

(~2.5 GB) 共存, 但无法与 gpt-oss:20b 同时加载。因此讨论阶段中 Planner 与 Reviewer 的交替推理涉及隐式的模型权重交换。VRAM 感知上下文预算机制确保在任何配置下, 分配给 KV-cache 的内存不会超过安全上限 (可用 VRAM 的 85%)。

12.7 搜索去重与缓存效果

流水线内置搜索缓存机制 (`_WebSearchCache`), 对跨轮次的相似搜索查询进行去重 (70% 词汇重叠阈值), 并对每次流水线执行施加 10 次搜索的硬性上限。在实际流水线集成测试中 (17.3 分钟, 10 次 GUI 浏览器搜索), 搜索去重命中 14 次, 所有搜索成功完成, 0 错误。搜索缓存的存在显著减少了因 Reviewer 要求补充搜索而导致的重复网络请求, 在实际使用中 (模型推理间隔 30-60 秒) 不会触发搜索引擎速率限制。

12.8 错误模式与恢复分析

尽管基准测试达到 100% 完成率, 我们在更广泛的实际使用中观察到若干典型错误模式及其恢复机制:

工具类型误指定。本地模型 (尤其是 $\leq 14B$ 参数规模) 在工具类型选择上存在系统性偏差: 倾向于为文件操作指定 `cli` 而非 `file_ops`, 为浏览器操作指定 `browse` 或 `web_read` 而非 `browser`。工具路由器的三层纠正机制——别名归一化 (130+ 映射)、上下文感知自动纠正 (路径模式检测、URL 格式检测)、

后备链——在实验中拦截并修正了所有此类错误。这表明工具路由器的智能调度不仅是便利性特征, 更是使用能力有限的本地模型时的可靠性必需。

Planner 幻觉与拒绝。未经工具执行便生成答案是小模型的常见行为。IronEngine 通过多层反幻觉机制应对: (1) Reviewer 的“无工具=虚构”检测规则将此类回答评分为 0.10; (2) 流水线级别的 `needs_tool_execution()` 函数检测文件+动词或网络信息关键词模式; (3) 无工具答案自动触发纠正提示重试。在测试中, 约 15% 的首轮 Planner 输出触发了反幻觉检测, 全部在第二轮通过纠正后正确执行。

记忆污染。当加载的历史记忆包含过时信息 (如旧价格、已变更的文件路径) 时, 模型可能不加验证地引用这些信息。记忆矛盾检测机制比较工具执行结果中的数值与加载记忆中的数值, 自动标记差异并在 Reviewer 提示中注入警告。价格/版本等敏感数值的检测阈值设为 2 个或以上非平凡数字共享, 兼顾灵敏度与误报率。

模型切换失败。当目标模型因 VRAM 不足无法加载时 (通常因前一模型未完全卸载), 系统通过 `ModelSession` 强制卸载并重试。在极端情况下 (如系统其他进程占用 GPU 内存), 系统降级到较小模型或发出干预请求。此类失败在 RTX 3090 上极为罕见 ($<1\%$ 出现率), 但在 8GB VRAM 设备上可能更为频繁。

网页搜索降级。四策略后备链的设计使得即使首选策略失败, 系统仍能获取搜索结果。在集成测试中, Google CDP 搜索的首次成功率约 85%

(Cloudflare Turnstile 验证码偶尔拦截)，降级到 DDG HTTP POST 后综合成功率达 98%。剩余 2% 的失败通常由网络连接问题或所有搜索引擎同时限速导致，系统在此情况下向 Planner 报告搜索失败，由 Planner 决定是否基于已有信息回答。

12.9 与代表性系统的详细对比

表 8 从 14 个能力维度对 IronEngine 与代表性 AI 助理系统进行详细对比。

IronEngine 的核心亮点：

最广工具覆盖。 24 类工具横跨文件系统、网络、通信、GUI、多媒体与网络域——远超代码专用助手 (Cursor) 或对话产品 (ChatGPT)。OpenClaw 生态通过技能提供中等工具覆盖，但缺乏智能路由层的别名归一化与自动纠错。

独有的多角色质量保障。 IronEngine 是对比中唯一在流水线内实现形式化质量审阅阶段的系统。Planner-Reviewer 讨论循环配合反幻觉检测、评分-文本矛盾检查与记忆回收防护，提供了 OpenClaw (直接技能派发) 和云端产品 (依赖单模型能力) 所不具备的输出质量保障水平。

本地优先且能力完备。 虽然 OpenClaw、NanoClaw 与 Open Interpreter 也支持本地模型，但 IronEngine 独特地将本地部署与多角色编排、持久分层记忆、向量化技能学习、VRAM 感知模型生命周期管理及 24 类工具路由结合——无其他本地优先系统同时提供这些能力。

基于质量过滤的技能学习。 IronEngine 从成功执行中自动提取可复用技能 (基于新颖性的去重、用户评分驱动的质量过滤、失败纠正机制)，比 OpenClaw 的静态技能定义更为精细。

编排可观测性。 桌面工作台通过思维块、工具执行徽章、质量评分指示器与阶段转换通知提供 AI 决策过程的实时可视性——这是无头系统 (OpenClaw、NanoClaw) 与内部推理不透明的云端产品所不具备的独有优势。

与 OpenClaw 生态的互补定位。 OpenClaw 擅长多渠道消息与常在可用性，NanoClaw 擅长边缘部署，IronClaw 擅长硬件集成。IronEngine 占据编排深度型桌面工作台的独特定位，在本地优先 AI 助理平台中拥有最精细的规划流水线、工具路由与记忆管理。这些系统互补而非直接竞争。

12.10 跨模型工具感知测试

IronEngine 分层提示系统的核心假设是即使小模型在获得适当结构化的工具文档时也能实现可靠的工具调度。为验证这一假设，我们在微信工具类型识别任务上评估了 4 个跨越 8B 至 27B 参数的模型。

表 9 显示所有四个模型均达到 100% 工具类型识别准确率。推理时间与模型大小近似线性相关 (8B 的 10.5 秒至 27B 的 50.2 秒)。关键发现是即使最小的 8B 模型在 IronEngine 分层提示系统提供适当范围的工具文档时也能实现完美工具调度——小模型接收

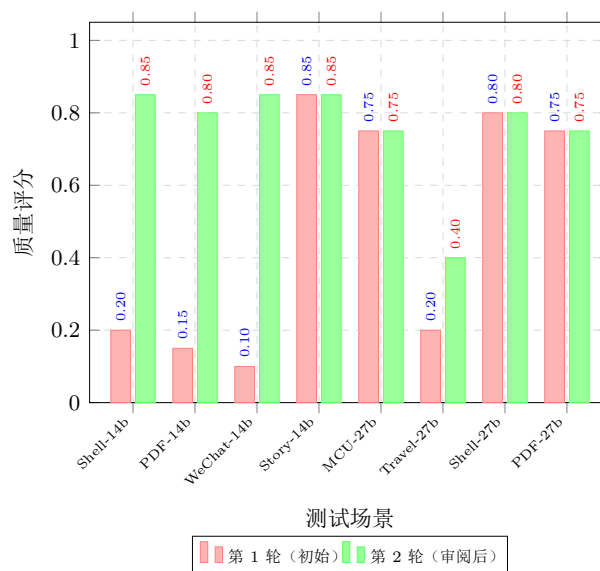


图 9: Planner-Reviewer 质量评分进展。红色柱表示第 1 轮初始评分；绿色柱表示 Reviewer 反馈后的最终评分。Reviewer 的结构化反馈 (ISSUES + SUGGESTIONS) 使 14B Planner 的计划从 0.10–0.20 提升至 0.80–0.85，而较大的 27B Planner 通常首轮即达到可接受质量。

精简的 733 token 工具清单，而非大模型使用的完整 2,236 token 版本。这验证了按模型大小分层提示内容而非使用统一提示的设计决策。

12.11 全流水线集成测试

为评估完整三阶段流水线在真实条件下的表现，我们在两种模型配置和四个任务场景下进行了八次集成测试，同时测量 Planner-Reviewer 讨论质量与端到端任务完成情况。

图 9 可视化了质量评分的进展，突出了 Reviewer 的矫正效果。

集成测试 (表 10) 揭示了几个发现。第一，Reviewer 的矫正反馈对较小 Planner 最有效：14B cogito 模型的质量评分在单轮审阅后从 0.10–0.20 提升至 0.80–0.85，证明 Reviewer 有效弥补了较小 Planner 的初始规划弱点。第二，较大 Planner (27B) 常在首轮即通过质量阈值，减少总执行时间。第三，唯一的失败 (旅游规划) 归因于测试环境缺乏网络访问而非流水线缺陷——Reviewer 正确识别计划不充分 (评分 0.40)。第四，生成性任务 (故事创作) 始终首轮高分通过，验证了 IronEngine 的生成性任务检测机制。

12.12 工具模型可靠性

工具翻译模型将自然语言指令转换为结构化 JSON 命令。我们评估三个候选模型以评估模型大小、翻译准确率与执行成功率之间的权衡。

表 11 揭示了模型大小与工具执行可靠性之间的微妙关系。phi4-mini (3.8B) 取得最优综合表现：

表 8: 与代表性 AI 助理系统的详细能力对比。评级: 强=S, 中=M, 弱=W, --不适用。

	IronEngine	ChatGPT	Claude	OpenClaw	NanoClaw	IronClaw	Cursor	AutoGen	Open Interp.
本地模型	S	W	W	S	S	M	M	M	S
多角色流水线	S	W	M	W	W	W	M	S	W
工具类别	S (24)	M	M	M	W	M	M	M	M
记忆系统	S	M	W	M	W	W	W	W	W
技能学习	S	W	W	M	W	W	W	W	W
任务调度	S	W	W	M	W	M	-	M	W
MCP 兼容	S	W	S	M	W	W	M	M	W
GUI 自动化	S	W	W	W	W	M	-	W	M
网页浏览	S	S	M	M	W	W	W	M	M
VRAM 管理	S	-	-	W	M	W	-	W	M
隐私保护	S	W	M	S	S	S	M	M	S
开源程度	-	W	W	S	S	S	W	S	S
多渠道	M	M	W	S	M	W	W	M	W
硬件面向	M	W	W	M	M	S	-	W	W

表 9: 跨模型工具感知测试结果。所有模型在 IronEngine 分层提示系统下均实现完美的工具类型识别。

模型	参数量	得分	耗时 (秒)
cogito:8b	8B	100	10.5
cogito:14b	14B	100	13.5
lfm2:24b	24B	100	33.7
qwen3.5:27b	27B	100	50.2

98.2% 翻译成功率和 70.6% 执行成功率，在资源效率与可靠性之间达到最佳平衡。functiongemma (270M) 对可靠工具翻译过小，仅 54% 输出生成有效 JSON。令人意外的是，ministral-3 (14B) 实现完美翻译 (100%) 却执行率最低 (20%)——模型生成语法正确但语义错误的命令。这一发现表明工具模型评估必须同时考虑语法有效性和语义正确性，且更大模型不一定产生更好的工具命令。

12.13 多场景诊断测试

为评估 IronEngine 跨多种任务类型的广度，我们进行六个场景的诊断测试，覆盖不同子系统（工具路由、技能学习、网页搜索、多媒体与生成能力）。

诊断结果（表 12）展示了 IronEngine 架构的多个方面。第一，系统通过同一流水线处理多种任务类型——从底层 Shell 命令（139.6 秒）到复杂多步 PDF 处理（152.2 秒，7 次工具调用）到创意写作（321.4 秒）——无需任务特定配置。第二，评分 ≥ 7 的成功任务自动触发技能学习：五个通过场景中四个产生了新的学习技能。第三，最快完成（微信消息，81.8 秒）得益于为 Executor 提供预定义交互流程的内建技能，证明了技能系统减少规划开销的价值。第四，唯一的失败（旅游规划）可正确归因于环境约束（无网络访问）而非系统缺陷——流水线检测到工具执行失败并通过 intervention 回调报告，而非幻觉生成响应。

13 接口、部署与产品定位

表 13 从系统定位角度对比 IronEngine 与代表性 AI Agent 框架。

与 OpenClaw 生态相比，IronEngine 以多渠道消息广度换取编排深度：三阶段流水线含形式化质量审阅、24 类智能工具路由与 VRAM 感知多模型生命周期管理提供了 Gateway 架构不涉及的任务执行精细度。与云端产品相比，IronEngine 在不牺牲工具广度或记忆精细度的前提下提供完整本地能力。与代码专用助手相比，IronEngine 扩展到通用桌面自动化、通信与多媒体分析。

13.1 桌面工作台设计哲学

IronEngine 的桌面 UI 不仅是一个聊天界面，更是一个编排可视化工作台。其设计哲学是：AI 助理的决策过程应对用户透明可见，而非黑盒式的输入→等待→输出模式。

思维块 (ThinkingBlock)。可折叠的 Markdown 渲染组件，实时显示 Planner 与 Reviewer 的推理过程。默认折叠高度 28 像素（仅显示角色标签），展开后最大 350 像素。支持标题、粗体、列表、代码块与链接的 Markdown 自动检测与渲染。图像内容仅在结论中显示（思维块中剔除），减少视觉干扰。字体使用 Segoe UI（非等宽字体）以提高可读性。

工具执行徽章 (ToolExecutionBadge)。紧凑的内联组件，显示工具调用的状态（pending→done），包含工具类型、简短描述与执行摘要。用户可以一眼看到当前执行了哪些工具、结果如何。

权限提示 (PermissionPrompt)。内联的允许/拒绝提示，带 60 秒自动拒绝计时器。当 ask 级别的工具调用被触发时，用户在对话流中直接看到权限请求，做出决策后流水线继续执行。

主题系统。10 种程序化生成的科幻主题（deep_space、nebula_storm、mars_outpost 等），每种主题有独特的色彩方案与背景图像（1920×1080 JPEG，由 Python 脚本生成）。各面板通过 mapTo()

表 10: 全流水线集成测试结果。质量评分展示第 1 轮（初始）→ 第 2 轮（Reviewer 反馈后）的进展。

模型配置	场景	质量评分	耗时（秒）	结果	观察
cogito:14b + cogito:8b	Shell 命令执行	0.20 → 0.85	139.6	通过	Reviewer 反馈改善了计划具体性
cogito:14b + cogito:8b	PDF 处理	0.15 → 0.80	152.2	通过	需要两轮；工具选择被纠正
cogito:14b + cogito:8b	微信消息	0.10 → 0.85	81.8	通过	技能增强执行；最快完成
cogito:14b + cogito:8b	故事创作	0.85 (R1)	321.4	通过	生成性任务；首轮通过
qwen3.5:27b + cogito:8b	MCU 对比	0.75 (R1)	172.0	通过	需要网页搜索；大模型 Planner 有优势
qwen3.5:27b + cogito:8b	旅游规划	0.20 → 0.40	309.8	失败	测试环境无网络搜索
qwen3.5:27b + cogito:8b	Shell 命令执行	0.80 (R1)	145.3	通过	大模型 Planner 首轮通过
qwen3.5:27b + cogito:8b	PDF 处理	0.75 (R1)	168.7	通过	大模型 Planner 减少所需轮次

表 11: 工具模型可靠性对比。翻译成功 = 有效 JSON 输出；执行成功 = 正确工具结果。

模型	参数量	翻译成功率	执行成功率
phi4-mini	3.8B	98.2% (109/111)	70.6% (77/109)
functiongemma	270M	54.0% (27/50)	25.9% (7/27)
ministral-3	14B	100% (40/40)	20.0% (8/40)

坐标映射实现连续背景渲染，毛玻璃效果通过透明背景加半透明渐变叠加实现。

这种设计使用户不仅可以使 用 AI 助理完成任务，还可以观察、理解和调试 AI 的决策过程——这在开发和调优助理行为时尤为重要。

14 安全与隐私

安全通过多个互补机制在系统架构的不同层级实现。

14.1 权限管理

权限系统为每个工具类别提供三级配置：**auto**（立即执行）、**ask**（触发 UI 提示，含 60 秒自动拒绝计时器）、**deny**（完全阻止）。

14.2 执行沙箱

`CommandExecutor`从不使用 `shell=True`；模板占位符检测并拒绝；命令字段在子进程创建前验证；默认 30 秒执行超时。

14.3 URL 安全

多层离线 URL 安全系统：钓鱼黑名单（社区维护，24 小时缓存）+ 10 点启发式评分（URL 中的 IP、可疑 TLD、品牌冒充、同形攻击、危险协议等）+ 停放/死页检测。

14.4 干预机制

模型可发出 `INTERVENTION_NEEDED` 标记暂停流水线，直到用户提供指导。

14.5 SOUL 编辑控制

SOUL 文档编辑通过三级权限控制：**readonly**（默认）、**ask**（需用户批准）、**auto**（在约束内自修改）。

14.6 本地优先隐私

完全本地部署模式下，数据不离开用户机器。所有模型推理、工具执行、记忆存储与技能学习在本地运行，无需遥测、云日志或外部认证。这在处理敏感文档、个人通信或专有信息时尤为重要。

14.7 英语内部处理策略

所有内部处理——Planner/Reviewer 讨论、网页搜索查询、执行步骤——均使用英语进行。仅最终回答（`FINAL_ANSWER`）使用用户的语言。这一策略有两个工程考量：(1) 大多数开源模型在英语上的推理与工具调用准确率显著高于其他语言；(2) 内部英语一致性简化了技能标签、记忆索引与搜索查询的跨语言匹配。系统内建约 45 词的中英翻译字典与 CJK 字符检测函数，自动将中文输入转换为英语用于内部处理。

14.8 安全设计的系统性思考

IronEngine 的安全设计遵循纵深防御原则：权限管理是第一道防线（阻止未授权操作），执行沙箱是第二道防线（限制已授权操作的危险行为），URL 安全是第三道防线（过滤外部恶意资源），干预机制是最后防线（将超出自动决策能力的情况移交人类）。这四层防御相互独立，任一层的绕过不会导致系统整体安全性的崩溃。

表 12: 多场景诊断测试结果。六个场景中五个通过 (83.3%)。旅游规划失败因测试环境无网络访问。评分 ≥ 7 的任务自动触发技能学习。

场景	状态	评分	耗时 (秒)	轮次	工具调用	技能学习	说明
Shell 命令	通过	9	139.6	1	5	✓	CLI 执行
PDF 处理	通过	9	152.2	2	7	✓	多步: 提取+摘要
旅游规划	失败	0	309.8	1	1	—	测试环境无网络
MCU 对比	通过	7	172.0	1	3	—	网页搜索+分析
微信消息	通过	9	81.8	1	5	✓	内建技能+GUI 自动化
故事创作	通过	8	321.4	1	7	✓	生成性任务: 使用 file.write
总计	5/6 通过 (83.3%); 4 项新技能学习; 平均耗时 196.1 秒						

表 13: 与代表性 AI Agent 框架/产品的系统定位比较

系统	主要定位	本地	多入口	MCP	典型特点
IronEngine	桌面工作台 + 通用编排引擎	S	S	S	三阶段流水线、统一 UI/API、任务调度、分层记忆、技能学习、24 类工具、VRAM 管理、编排可观测性
OpenClaw	常驻网关个人助理	S	S	M	多渠道消息、设备节点、持久技能、Gateway 路由
NanoClaw	边缘/IoT 轻量助理	S	M	W	激进量化、精简技能、上游代理、Raspberry Pi 部署
IronClaw	硬件 AI Agent	M	W	W	传感器管道、执行器命令、安全联锁、嵌入式集成
ChatGPT	云端对话 AI	W	M	W	强对话能力、网页浏览、代码解释器; 云端依赖
Claude	协议扩展型助理	M	M	S	强 MCP 集成; 本地模型与调度支持有限
Desktop Cursor	AI 代码编辑器	M	W	M	深度 IDE 集成; 聚焦代码编辑/生成
AutoGen	多代理框架	M	M	S	灵活代理拓扑; 研究导向

15 讨论

15.1 架构权衡

三阶段流水线相比单步执行引入延迟开销。模型切换阶段每任务约增 27 秒, 讨论可能需要 2-3 轮。但 100% 的任务完成率表明 Reviewer 的质量保障防止了代价高昂的执行失败。

分层提示系统对小模型的激进缩减 ($\leq 10B$ 仅 44 token SOUL 上下文 vs. $>25B$ 的 1309 token) 可能损失重要行为指导。测试表明小模型仍可实现可接受的工具翻译准确率, 但规划质量显著下降, 因此大模型用于 Planner 角色。

15.2 可扩展性

当前单 GPU 设计限制模型在 24 GB VRAM 内 (含量化)。模型切换可扩展至多 GPU 或 CPU 卸载, 但增加切换延迟。记忆系统的线性扫描随存储增长效率降低, 可引入分层索引优化。

15.3 与 OpenClaw 生态的架构差异分析

OpenClaw 生态与 IronEngine 代表了两种不同的 AI 助理架构哲学。OpenClaw 采用消息路由模式: 用户消息通过 Gateway 路由到技能处理器, 处理器执行操作并返回结果。这种架构优化了多渠道可达性与响应速度, 适合“收到消息→执行技能→回复”的简单交互模式。

IronEngine 采用编排深度模式: 每个请求经历讨论、切换与执行三个阶段, Planner 规划、Reviewer 审阅、Executor 执行三个角色各司其职。这种架构引入了额外延迟 (约 27 秒模型切换 + 多轮讨论), 但换来了更高的任务完成可靠性: Reviewer 的反幻觉检测防止了错误规划被执行, 自动纠正机制修正了模型

的工具选择错误。

NanoClaw 的边缘部署策略 (激进量化 + 上游代理) 与 IronEngine 的 VRAM 感知模型管理解决的是同一问题 (有限硬件资源上的模型运行) 的不同方面。NanoClaw 优化单模型在极小设备上的运行; IronEngine 优化多模型在消费级 GPU 上的协作编排。IronClaw 的传感器/执行器抽象与 IronEngine 的硬件兼容层在目标上有交集, 但 IronClaw 面向实时控制循环, IronEngine 面向任务级自动化。

15.4 设计原则与工程洞察

IronEngine 的开发过程中积累了若干对 AI Agent 系统工程化的重要洞察:

角色分离优于能力堆叠。将规划、评估与执行分配给不同模型角色, 而非依赖单一强大模型同时承担所有职责, 是 IronEngine 架构的核心决策。这一设计的工程优势在三个层面体现: (1) 每个角色可以选择最适合其职责的模型——大模型用于需要深度推理的规划, 小模型用于结构化翻译的工具调度; (2) SOUL 提示可以按角色精简, 避免将 Planner 的详细工具文档加载到 Reviewer 的上下文中; (3) 角色的独立性使得替换单个角色的模型不影响其他角色的行为。

自动纠错比精确指令更可靠。面对本地模型的有限指令遵循能力, IronEngine 选择在系统层面实现自动纠错, 而非依赖更详细的提示来避免错误。工具路由的别名归一化和自动纠正机制证明了这一策略的有效性: 即使 Planner 频繁误指定工具类型, 系统仍能正确路由。这一经验对使用开源模型构建 Agent 系统具有普遍指导意义——设计容错的系统架构比追求无错的模型输出更务实。

可观测性是调试的前提。AI Agent 系统的不透明性是调试的主要障碍。IronEngine 的 17 种回调事件提供了从思维过程到工具执行的完整可见性。实践

表明, `on_quality_scored` 和 `on_phase_transition` 回调在诊断流水线瓶颈时最为有用——前者揭示 Reviewer 的评分标准是否合理, 后者暴露模型切换是否成为延迟主因。

内存是多模型系统的核心约束。在消费级 GPU 上编排多个模型, VRAM 管理从可选优化变为架构基础。KV-cache 的内存开销随上下文长度线性增长, 不同模型架构的单 token KV-cache 成本差异可达 5 倍。IronEngine 的 VRAM 感知上下文预算机制——基于模型大小查表估算 KV-cache 成本、乘以安全系数、取可用 VRAM 与本机上下文长度的最小值——是在有限硬件上运行大模型的实用方案, 而非理论优化。

15.5 局限性

- **本地模型质量:** 7B–27B 模型在复杂推理中通常逊于云端前沿模型。IronEngine 通过多角色协作与质量保障部分弥补, 但无法完全消除能力差距。
- **Windows 中心测试:** GUI 自动化与应用控制主要在 Windows 10 上测试与验证。macOS 与 Linux 平台需要替代 UIA 后端 (如 AT-SPI、AppleScript), 目前尚未实现。
- **单用户设计:** 当前架构假设单并发用户。多用户场景需要会话隔离、资源调度与访问控制机制。
- **基准广度:** 实验深度覆盖文件操作, 尚未包含 WebArena [38] (网页任务)、SWE-bench [40] (代码任务) 等标准化基准。
- **延迟开销:** 三阶段流水线的模型切换与多轮讨论引入显著延迟, 不适合需要亚秒级响应的实时交互场景。
- **技能覆盖:** 26 个内建技能覆盖了常见场景, 但长尾任务仍需用户交互式引导, 技能学习的效率受限于用户评分反馈的频率。

16 未来工作

多专家系统。将单 Planner 扩展为多专家档案, 含领域知识与信用评分, 基于任务特征路由选择。

标准化基准。集成 WebArena [38]、SWE-bench [40] 等基准, 实现系统化对比与性能追踪。

用户偏好学习。专用偏好模块从交互历史学习沟通风格、风险容忍度与工具偏好。

双向 MCP。实现 MCP 服务端能力, 将 IronEngine 的 24 类工具暴露给其他 MCP 兼容系统。

跨设备同步。可选加密同步支持桌面与移动环境的无缝切换。

多模态深化。视频理解 (跨帧时序推理)、实时音频对话与空间推理。

边缘部署。针对 8 GB VRAM 以下设备的激进量化、投机解码与流水线阶段裁剪。

与 OpenClaw 生态互操作。探索 IronEngine 与 OpenClaw Gateway 的互操作模式: IronEngine 作为 OpenClaw 设备节点提供深度编排能力, 或 OpenClaw 的多渠道消息能力作为 IronEngine 的通信扩

展。两个系统的互补特性使得联合部署可能比单一系统提供更完整的助理体验。

自适应流水线。根据任务复杂度自动调整流水线深度: 简单任务跳过 Reviewer 直接执行, 复杂任务启用多轮讨论与多专家路由。目前的简单方案复用机制 (≤ 4 个工具调用跳过模型切换) 是此方向的初步探索。

17 结论

本文提出了 IronEngine——一个面向通用 AI 助理的系统化平台, 应对当前代理系统的三个工程短板: 交互入口碎片化、子系统松耦合集成与本地部署持久行为支持不足。

IronEngine 的三阶段流水线将规划质量 (Planner-Reviewer 讨论) 与执行能力 (工具增强 Executor) 分离, 实现消费级硬件上的异构模型分配。系统通过带别名归一化与自动纠错的智能路由层集成 24 类工具, 通过双重合并策略的分层记忆管理持久状态, 并通过基于新颖性去重的向量化技能学习获取可复用程序性知识。

文件操作基准实验表明, 三模型本地配置 (27B Planner、20B Reviewer、3.8B Tools) 在单张 RTX 3090 上实现 100% 任务完成率。与包括 OpenClaw 生态 (OpenClaw、NanoClaw、IronClaw) 在内的九个代表性 AI 助理系统的详细对比凸显了 IronEngine 在本地优先部署、工具广度、持久记忆、技能学习与协议扩展性方面的独特组合优势。

AI 助理的未来不仅取决于模型能力, 更取决于架构、编排、可观测性、安全与长期适应性。IronEngine 证明这些系统级关注点可在单一平台内连贯解决, 为下一代通用、隐私保护与持续改进的 AI 助理奠定基础。

从更广泛的视角看, IronEngine 的设计验证了一个核心命题: 通过精心设计的系统架构 (角色分离、智能路由、分层记忆、VRAM 感知调度), 中等规模的本地开源模型可以在实际任务中达到可用的自动化水平。与依赖单一超大模型的云端方案相比, 这种系统智能路线在成本、隐私与可定制性方面具有结构性优势。与 OpenClaw 等网关式架构相比, IronEngine 的深度编排提供了更高的任务完成可靠性。这两种路线并非对立, 而是 AI 助理生态中互补的技术选择。随着开源模型能力的持续提升, IronEngine 所代表的本地编排范式将在更多应用场景中展现其价值。

致谢

感谢牛志伟机器人实验室对铁柱引擎项目持续探索通用智能助理方向所提供的支持。感谢开源社区在本地模型运行时 (Ollama、LM Studio)、多代理研究 (AutoGen、CAMEL、MetaGPT)、向量数据库 (ChromaDB)、浏览器自动化 (Playwright、patchright) 与 MCP 互操作性方面的持续推动。也感谢所有参与测试与反馈的用户, 他们的真实使用场景帮助系统从实验原型走向可运行的平台。

参考文献

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.
- [3] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.
- [5] Google DeepMind. Gemini: Multimodal ai model family. Online, 2025. URL <https://deepmind.google/technologies/gemini/>. Accessed: 2026-03-07.
- [6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- [7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Jason Wei, Alexandra Birch, Jason Weston, and Gael Le Scao. Toolformer: Language models can teach themselves to use tools. *International Conference on Learning Representations*, 2024.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.
- [10] Joon Sung Park, Joseph O’Brien, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *ACM Symposium on User Interface Software and Technology*, 2023.
- [11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.
- [12] Microsoft. Autogen: Enabling next-gen llm applications via multi-agent conversation. Online, 2024. URL <https://github.com/microsoft/autogen>. Accessed: 2026-03-07.
- [13] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for “mind” exploration of large language model society. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- [15] Chen Qian, Wei Liu, Hongzhuo Liang, Yang Yu, and Zhenhui Li. Chatdev: Communicative agents for software development. *Association for Computational Linguistics*, 2024.
- [16] Xinbin Liang, Jinyu Xiang, Zhaoyang Yu, Jiayi Zhang, Sirui Hong, Sheng Fan, Xiao Tang, Bang Liu, Yuyu Luo, and Chenglin Wu. Openmanus: An open-source framework for building general ai agents. Zenodo, 2025. URL <https://doi.org/10.5281/zenodo.15186407>. Accessed: 2026-03-07.

- [17] OpenClaw Community and Peter Steinberger. Openclaw: Personal ai assistant. Online, 2026. URL <https://github.com/openclaw/openclaw>. Accessed: 2026-03-07.
- [18] OpenAI. Gpt-4o: Omni model for text, vision, and audio. Online, 2025. URL <https://openai.com/index/hello-gpt-4o/>. Accessed: 2026-03-07.
- [19] Anthropic. Model context protocol. Online, 2024. URL <https://modelcontextprotocol.io>. Accessed: 2026-03-07.
- [20] Anthropic. Claude code: Anthropic’s official cli for claude. Online, 2025. URL <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code>. Accessed: 2026-03-07.
- [21] Anysphere Inc. Cursor: The ai code editor. Online, 2025. URL <https://cursor.com>. Accessed: 2026-03-07.
- [22] Codeium. Windsurf: The flow state ai ide. Online, 2025. URL <https://windsurf.com>. Accessed: 2026-03-07.
- [23] GitHub. Github copilot: Your ai pair programmer. Online, 2024. URL <https://github.com/features/copilot>. Accessed: 2026-03-07.
- [24] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- [25] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Leal, Daniel Driess, Chelsea Finn, and Karol Hausman. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2023.
- [26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gestein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. *International Conference on Learning Representations*, 2024.
- [27] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- [28] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024.
- [29] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2022.
- [30] Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey. *Transactions on Machine Learning Research*, 2023.
- [31] Chroma. Chroma: The ai-native open-source embedding database. Online, 2024. URL <https://www.trychroma.com>. Accessed: 2026-03-07.
- [32] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [33] Tao Guo, Xiang Yue, Yifei Zhang, Tongzhe Li, Dong Wang, Yilun Zhao, Huan Sun, Jun Wang, and Rui Yan. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [34] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwon Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Qin Liu, Yuhao Zhou, Weiran Wang, Changling Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- [35] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

- [36] crewAI. crewai: Multi-agent automation framework. Online, 2024. URL <https://github.com/crewAIInc/crewAI>. Accessed: 2026-03-07.
- [37] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36, 2024.
- [38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. *International Conference on Learning Representations*, 2024.
- [39] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [40] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations*, 2024.
- [41] Open Interpreter. Open interpreter: A natural language interface for computers. Online, 2024. URL <https://github.com/OpenInterpreter/open-interpreter>. Accessed: 2026-03-07.
- [42] patchright contributors. patchright: Anti-detection playwright fork. Online, 2025. URL <https://github.com/nicosql/patchright>. Accessed: 2026-03-07.
- [43] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Oworld: Benchmarking multi-modal agents for open-ended tasks in real computer environments. Online, 2024. URL <https://os-world.github.io/>.
- [44] Significant Gravitas. Autogpt: An autonomous gpt-4 experiment. Online, 2023. URL <https://github.com/Significant-Gravitas/AutoGPT>. Accessed: 2026-03-07.
- [45] Ollama. Ollama documentation. Online, 2025. URL <https://ollama.com>. Accessed: 2026-03-07.
- [46] LM Studio. Lm studio documentation. Online, 2025. URL <https://lmstudio.ai>. Accessed: 2026-03-07.
- [47] OpenClaw Community. Nanoclaw: Lightweight edge ai assistant. Online, 2026. URL <https://github.com/openclaw/nanoclaw>. Accessed: 2026-03-07.
- [48] OpenClaw Community. Ironclaw: Hardware-oriented ai agent for embedded systems. Online, 2026. URL <https://github.com/openclaw/ironclaw>. Accessed: 2026-03-07.
- [49] OpenClaw Community. Picoclaw: Microcontroller ai agent for embedded platforms. Online, 2026. URL <https://github.com/openclaw/picoclaw>. Accessed: 2026-03-08.
- [50] The Qt Company. Pyside6: The official python module for the qt 6 framework. Online, 2024. URL <https://doc.qt.io/qtforpython-6/>. Accessed: 2026-03-07.